AI for Anomaly Detection

Lecture 6: Residual models: Convolutional Neural Networks

Lilach Goren Huber
Institute for Data Science (IDS)

zh aw School of Engineering





WPI AnoDet

#	Date	Topic
1	15.09	Introducation to anomaly detection (AD)
2	22.09	Supervised ML for AD
3	29.09	Unsupervied ML for AD I
4	06.10	Unsupervised ML for AD II
5	13.10	Residual models: reconstruction ML methods
6	20.10	Residual models: Convolutional neural networks (CNNs)
7	27.10	Residual models: regression models
8	03.11	Initial assignment presentations – obligatory presence in person
9	10.11	Residual models: advanced regression neural networks (CNNs, TCNs)
10	17.11	Domain shifts and data scarcity: transfer learning
11	24.11	Contamination and mislabeling
12	01.12	Quantifying uncertainty, physics informed ML
13	08.12	Recent advances in AD research
14	15.12	Final assignment presentations – obligatory presence in person

Admin

1. Fill in the class evaluation.

2. Next week: Kristupas Bajarunas

3. Check out the presentation schedule for the 3.11. Be ready at least 10 minutes before your time slot.

BA topics in our team

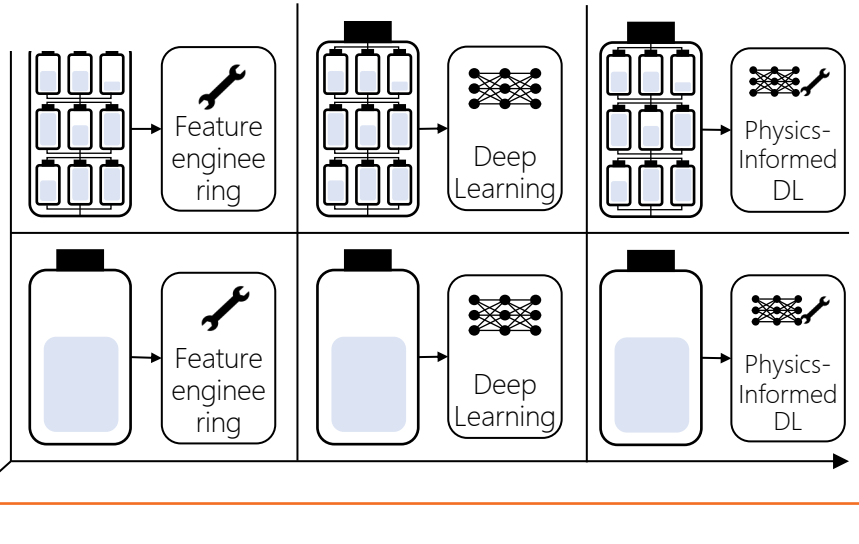
Optimal scheduling of condition based maintenance with Südostbahn.

Anomaly detection for train components with Stadler Services.





Intelligent equipment monitoring for clamping tools with Oetiker Schweiz.



Battery diagnostics and prognostics (potential for academic publication).

Today

Novelty detection with neural networks

- Recap NNs → Autoencoders
- Image AD → Intro to Convolutional Neural Networks
- Pretrained CNNs for feature extraction
- Example AD with pretrained CNNs
- Systematic NN development

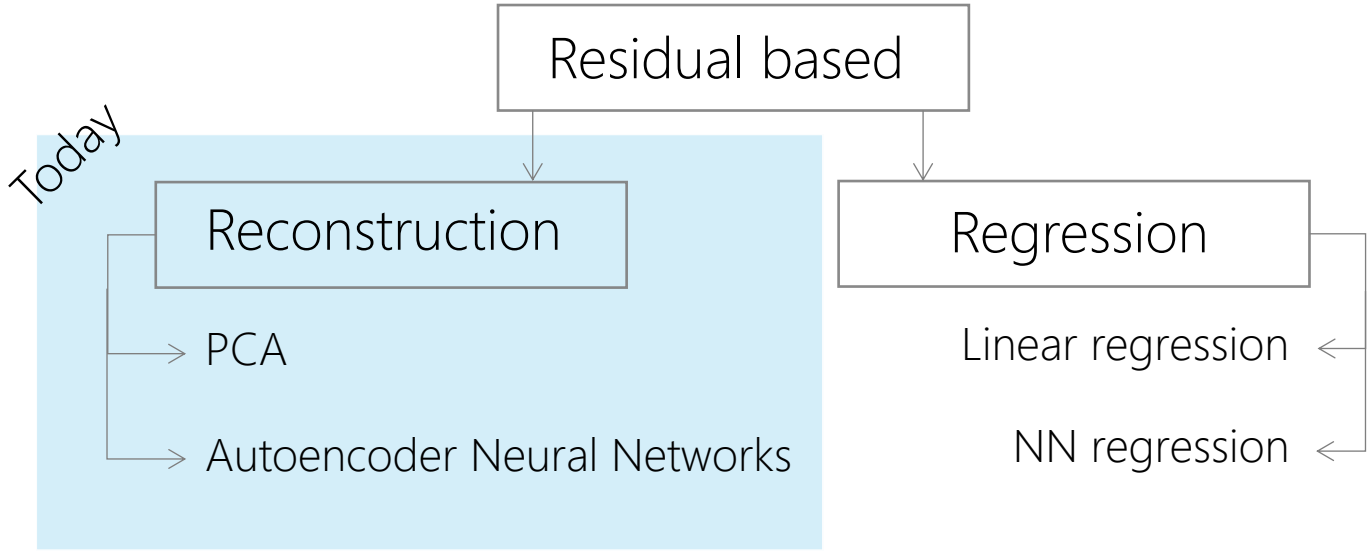
Novelty detection methods

Only normal data available for training → residual based approaches

1. Train with normal (healthy) data.
2. Use residuals to derive anomaly scores.
3. Select a threshold to detect abnormal samples.

Works with

- Tabular data
- Time series
- images



Recap neural networks

Homework: watch videos of 3Blue1Brown

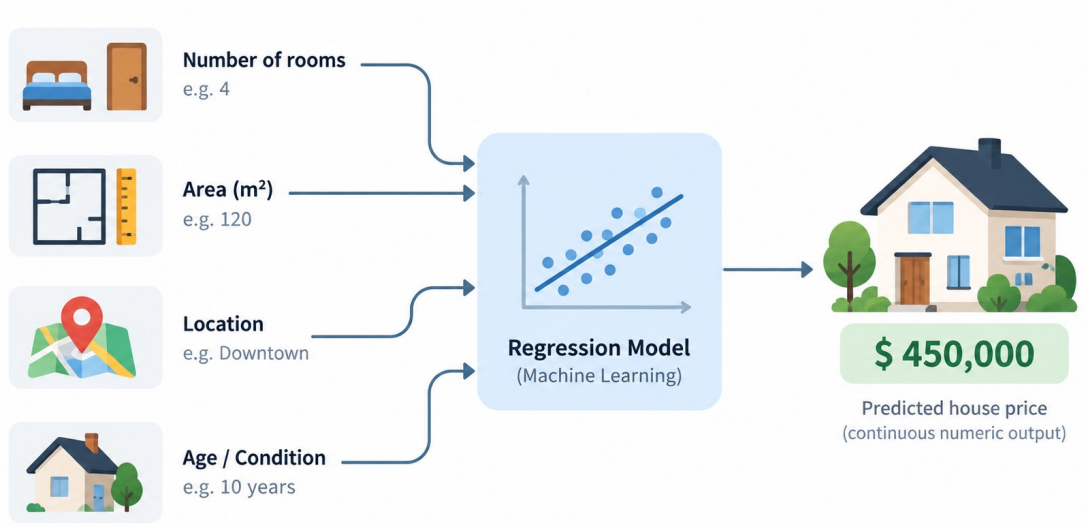
1. But what is a neural network?
2. Gradient descent, how neural networks learn
3. Backpropagation, intuitively
4. Backpropagation calculus

Recap neural networks

Neural networks are models that provide an output given an input.

When they are well trained they can give the correct output given an input they have never seen.

→They learn to generalize.

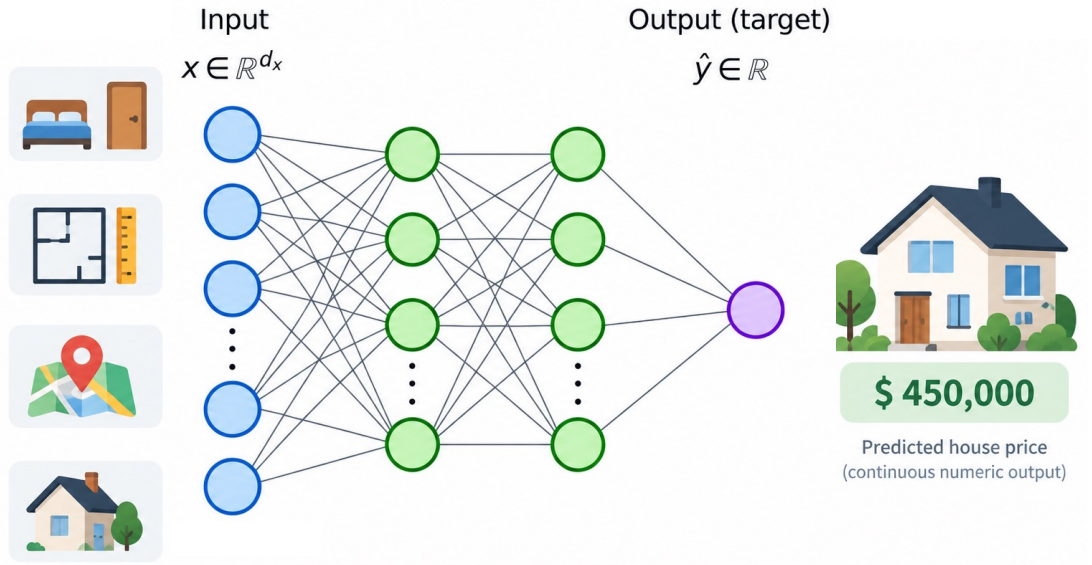


Recap neural networks

Neural networks are models that provide an output given an input.

When they are well trained they can give the correct output given an input they have never seen.

→They learn to generalize.



Recap neural networks

Neural networks are models that provide an output given an input.

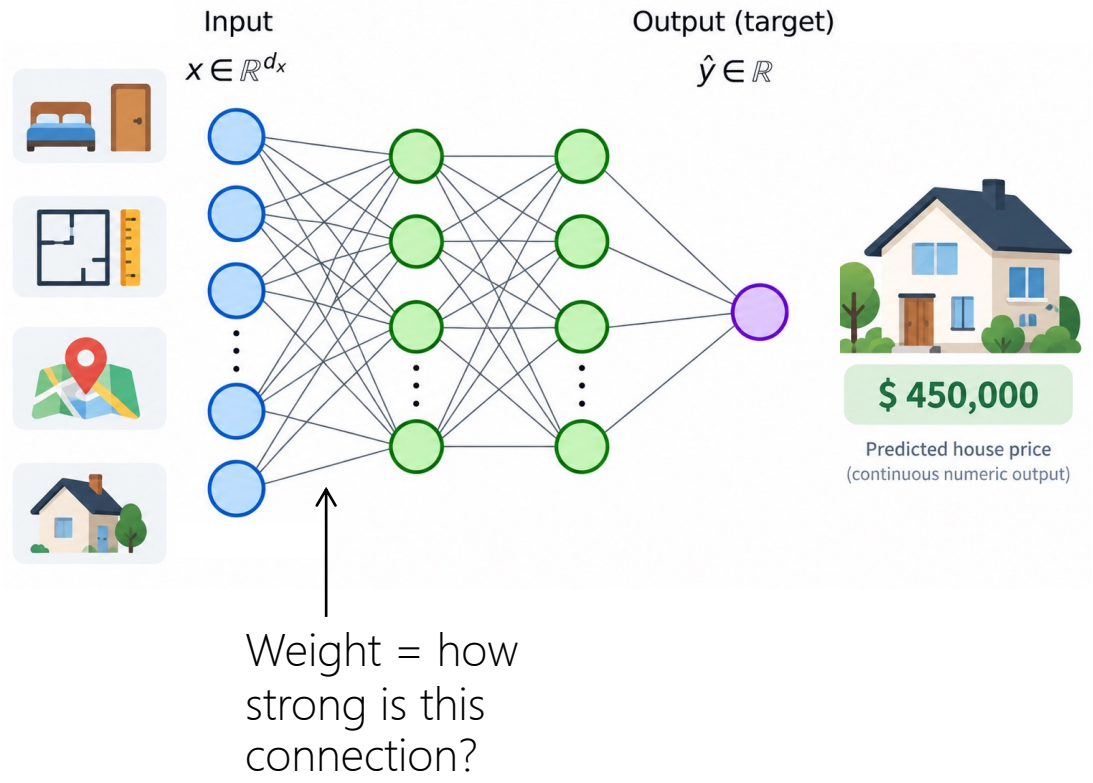
When they are well trained they can give the correct output given an input they have never seen.

→They learn to generalize.



Recap neural networks

How is learning achieved?
- By tuning the weights.



At the end of weight-tuning, all weights are optimal for this prediction task.

Recap neural networks

How does the network know in which direction to tune the weights?
- It has a loss function, and the weights are tuned such that the loss is minimized on the training data.

- Loss = cost.
- Non-optimal weights → high cost on the training data
- Optimal weights → lowest cost on the training data

Example: Loss function for house price prediction

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{\mathrm{N}} \sum_{\mathrm{i}=1}^{\mathrm{N}} |\boldsymbol{y_i} - \hat{\boldsymbol{y}}_{\boldsymbol{i}}|^2$$

MSE loss

Sum over training houses

Observed price

Predicted price

random weights

predictions far from observations (very wrong prices)

high loss

Training epochs

optimal weights

predictions near observations (accurate prices)

minimal loss

Recap neural networks

Training:

1. **Initialize** weights
2. **Forward pass:** Predict outputs for a batch of samples and calculate the loss.
3. **Backward pass:** Compute the gradients of the loss with respect to the weights (*backpropagation*).
4. **Update weights:** Use the gradients to reduce the loss (*optimizer*).
5. **Repeat** for many batches and epochs, monitoring training and validation loss.

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}_i|^2$$

- $\mathcal{L}_{MSE}$: MSE loss
- $\sum_{i=1}^{N}$: Sum over training houses
- y_i: Observed price
- $\hat{y}_i$: Predicted price

random weights

predictions far from observations (very wrong prices)

high loss

during training change weights to gradully lower the loss

Training epochs

optimal weights

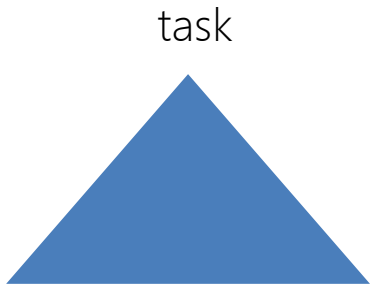
predictions near observations (accurate prices)

minimal loss

Recap neural networks

NNs are **models** trained with **data** to perform a **task**.

To generalize beyond the training samples we need:

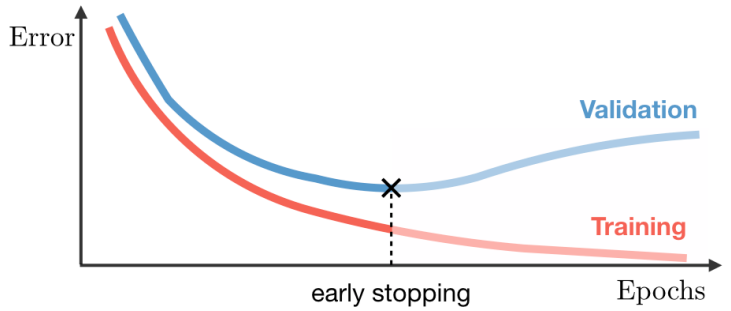


- Sufficient input-output examples
- Representative
- Reliable labels

- A suitable NN architecture
- A loss function aligned with the task
- The learning rate
- Training duration
- Regularization

How do we know if training succeeded?

1. **Learning curves**: Inspect training and validation loss for convergence and overfitting
2. Performance on unseen data: evaluate task-specific **metrics** on held-out validation data.
3. **Error analysis**: examine residuals or misclassifications and inspect examples with large errors.



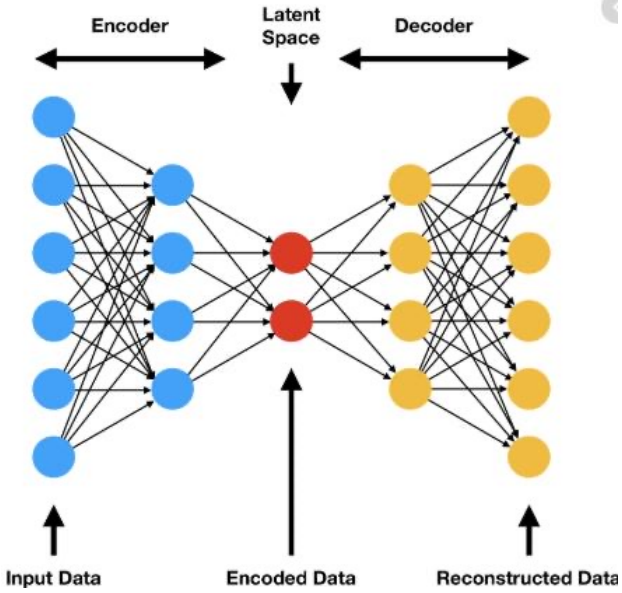
Autoencoder NN

Example: machine fault detection

Train an AE to reconstruct a vector of machine variables (temperature, pressure, motor current…).

Train on healthy machine data

$X \in \mathbb{R}^d$



$\hat{X} \in \mathbb{R}^d$

Update weights to minimize the reconstruction loss

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{\mathrm{N}} \sum_{\mathrm{i}=1}^{\mathrm{N}} \left| X_i - \hat{X}_i \right|^2$$

MSE loss

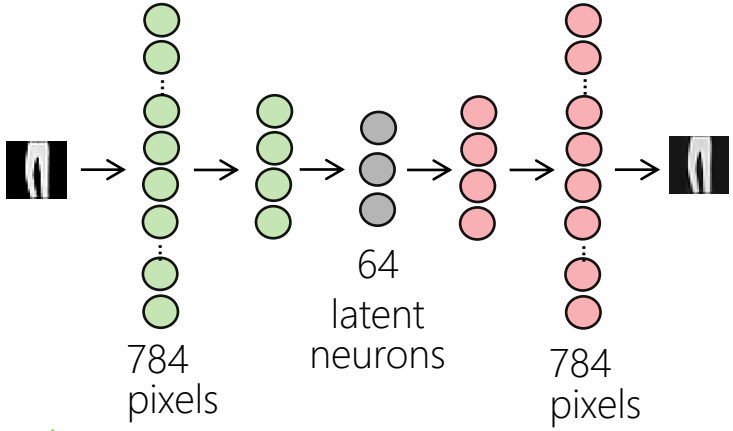
Sum over training samples

Observed machine variables

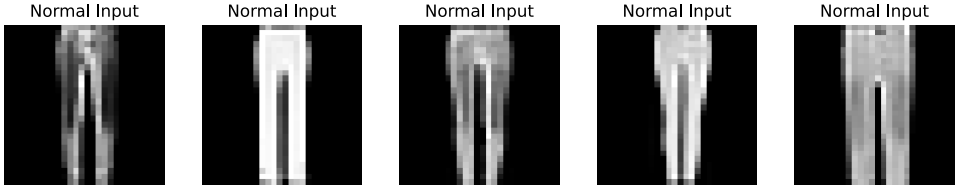
Reconstructed machine variables

Image Anomaly Detection with an Autoencoder

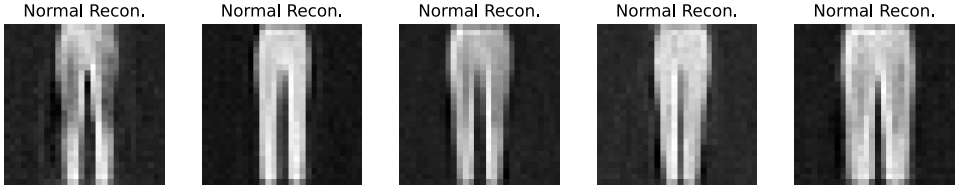
Example: trousers vs. all



Normal images

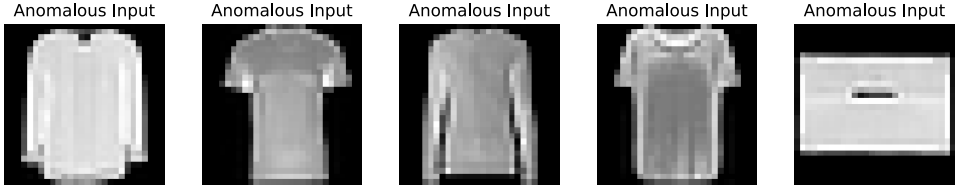


input

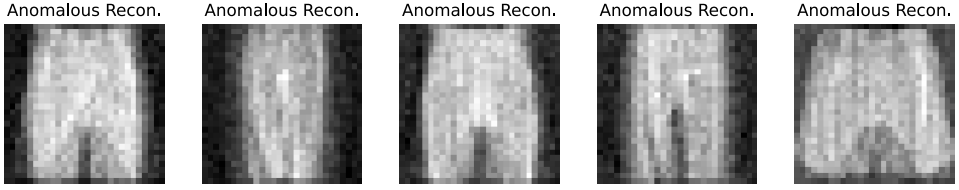


reconstruction

Abnormal images



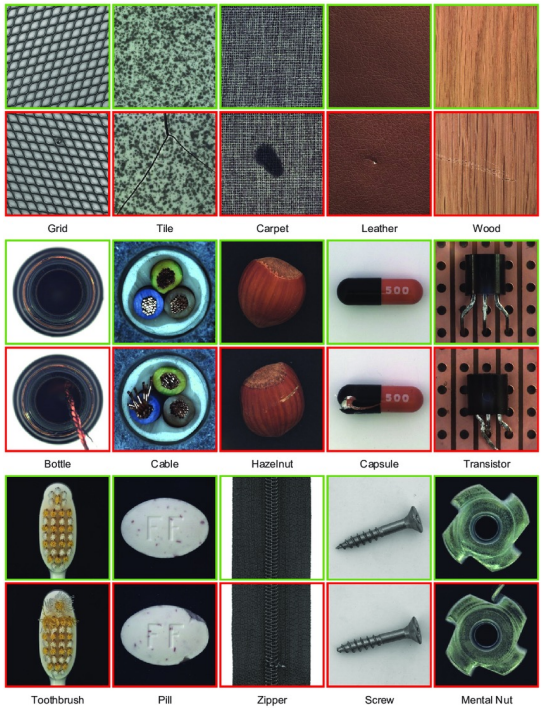
input

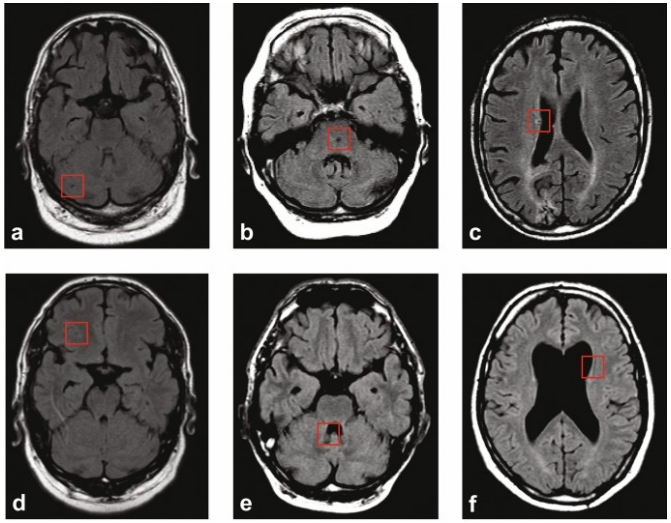


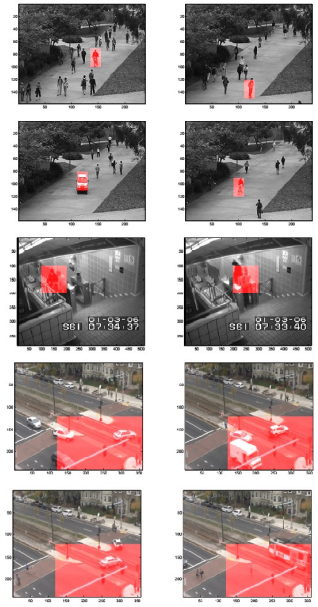
reconstruction

Image Anomaly Detection with NNs

- Feature extraction in images: Convolutional Neural Networks
- The extracted features can be used for any downstream task:
  - Classification
  - Regression
  - Reconstruction

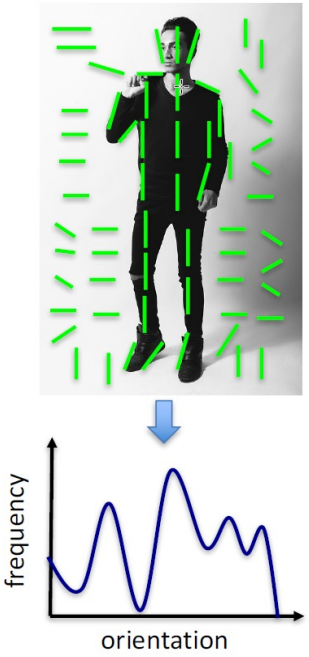






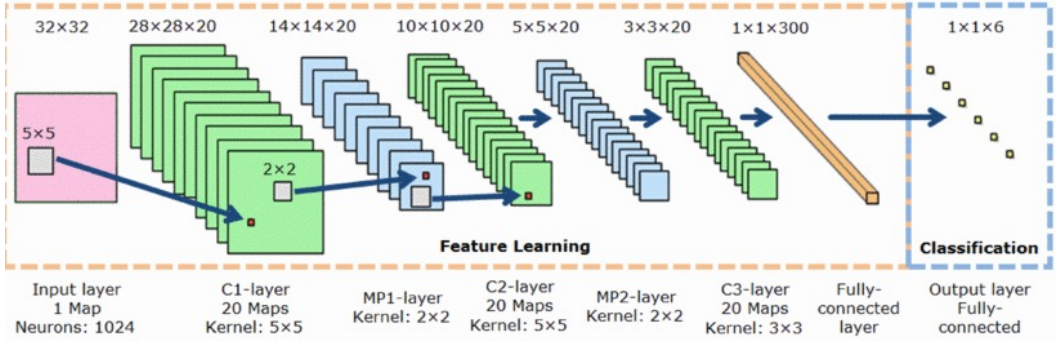
Paradigm Shift

Before 2012:
Hand-crafted features → classifier



After 2012:

- End-to-end feature learning from raw pixels
- Features are no longer engineered: they are learned automatically.



Simple Black & White Example: Classify X or O

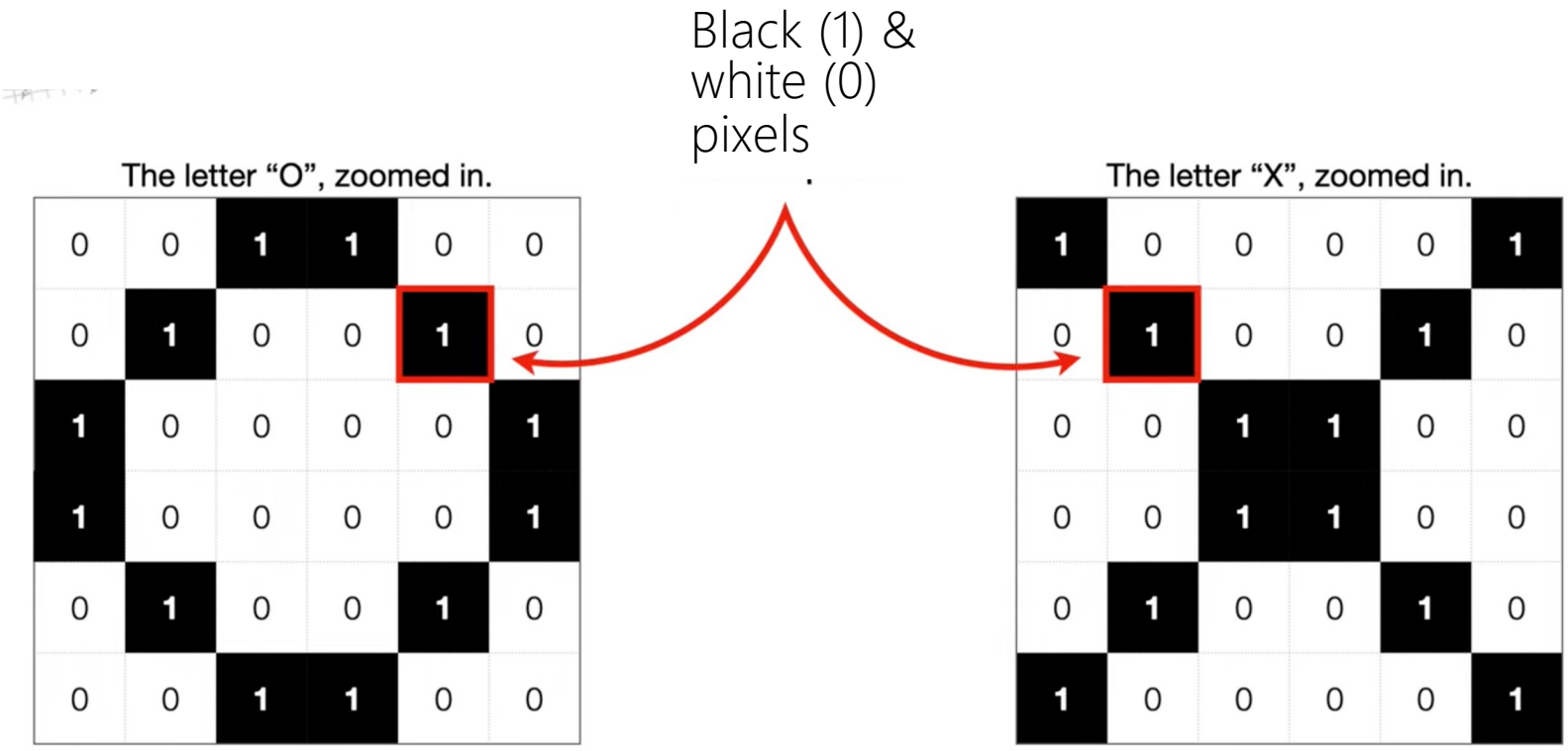


Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

Use a Fully Connected Neural Network?

Works for small images (6x6).

For 1000x1000 → 10^12 parameters.

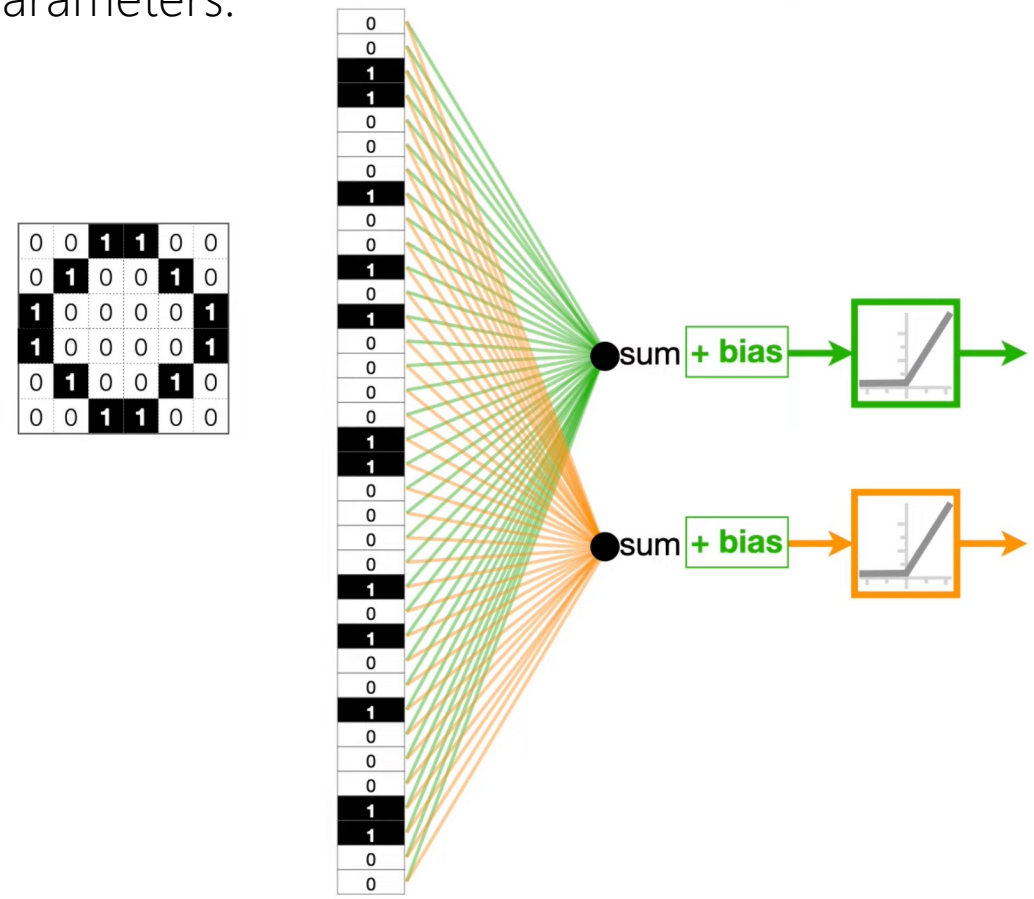
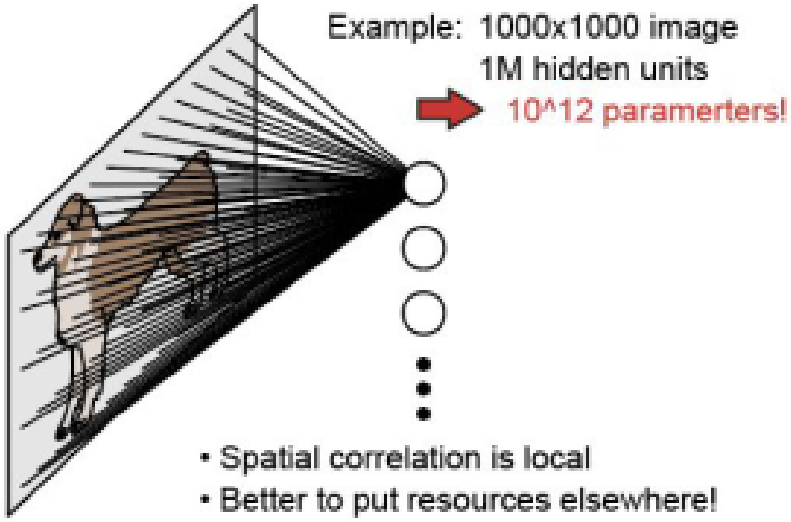
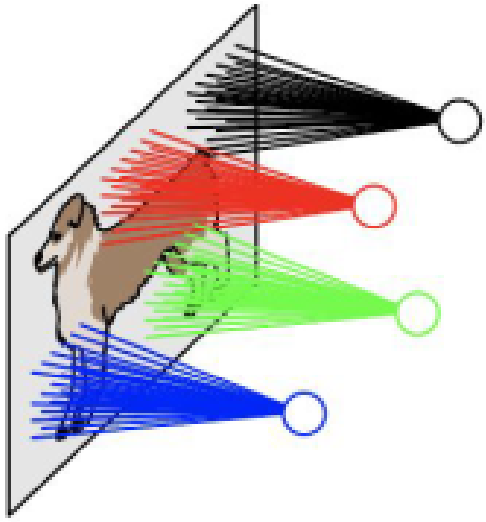


Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

The idea of CNN: share weights efficiently





FCNN:
weights connect every two pixels.

CNN:
use the same weights for every patch
→ capture local correlations
→ extract local features (edges, corners…) with less parameters.

Use a FCNN?

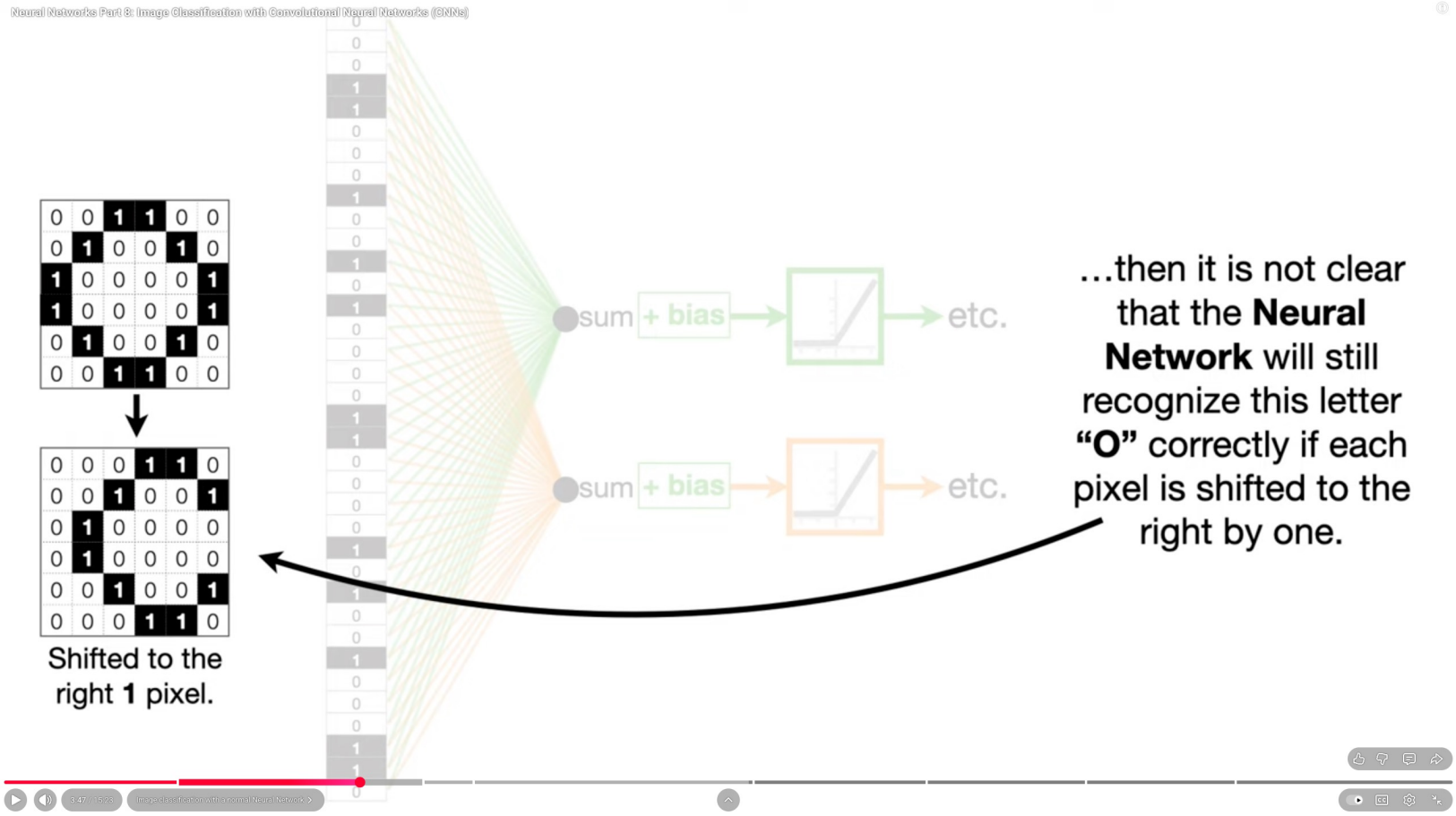


Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

Use a FCNN?

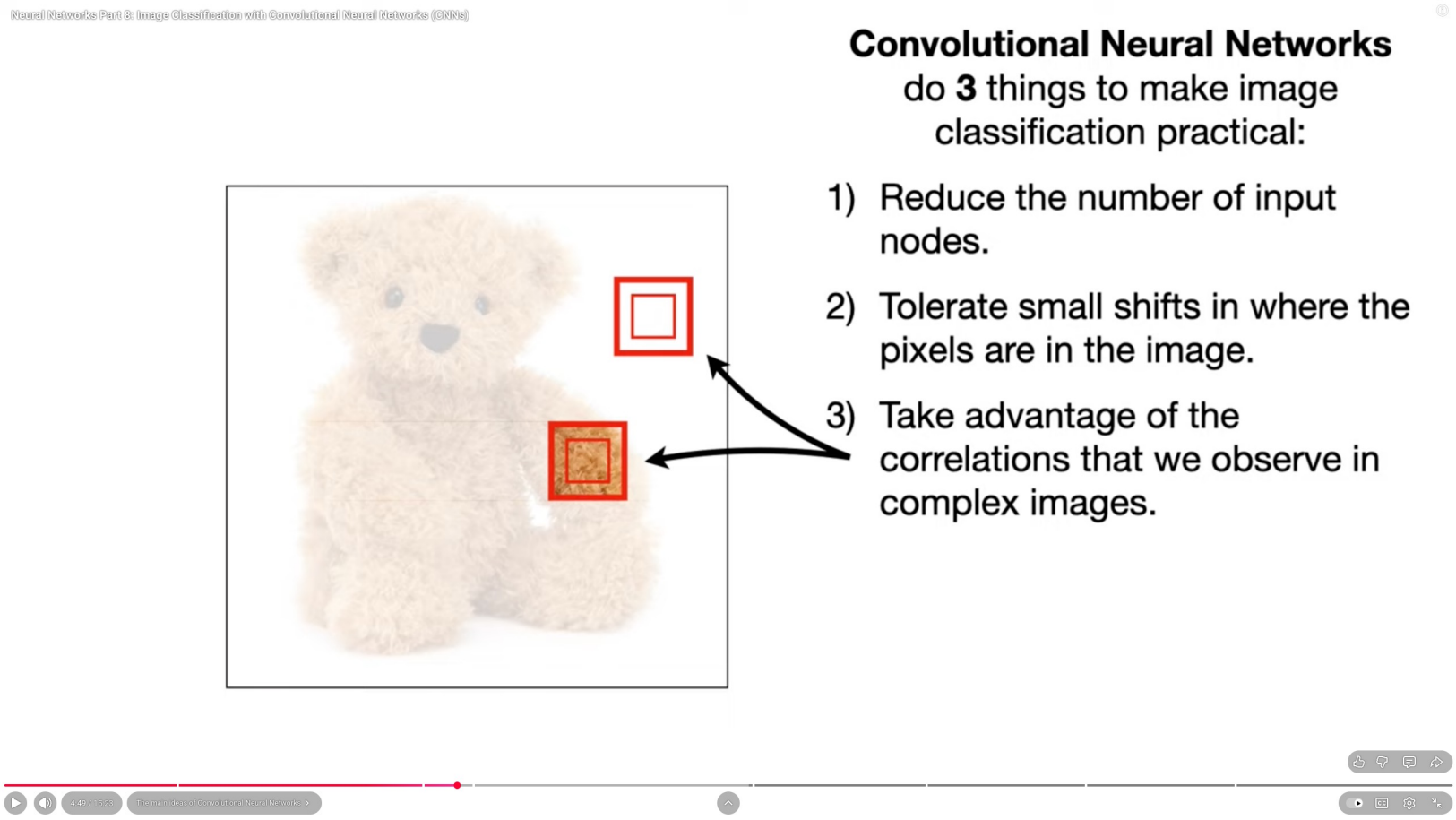
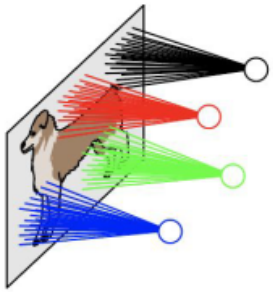


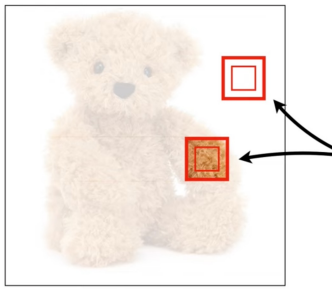
Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

Why not FCNN?

- Scaling: need p^4 weights for a p x p pixels image



- Spatial correlations: nearby pixels are likely to be similar



- Shift invariance: not clear that a FCNN would manage to classify a shifted image correctly

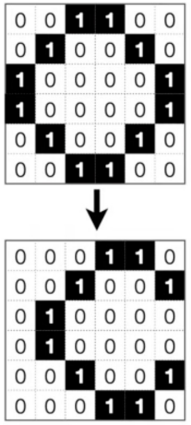


Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

CNNs use filters instead of single weights

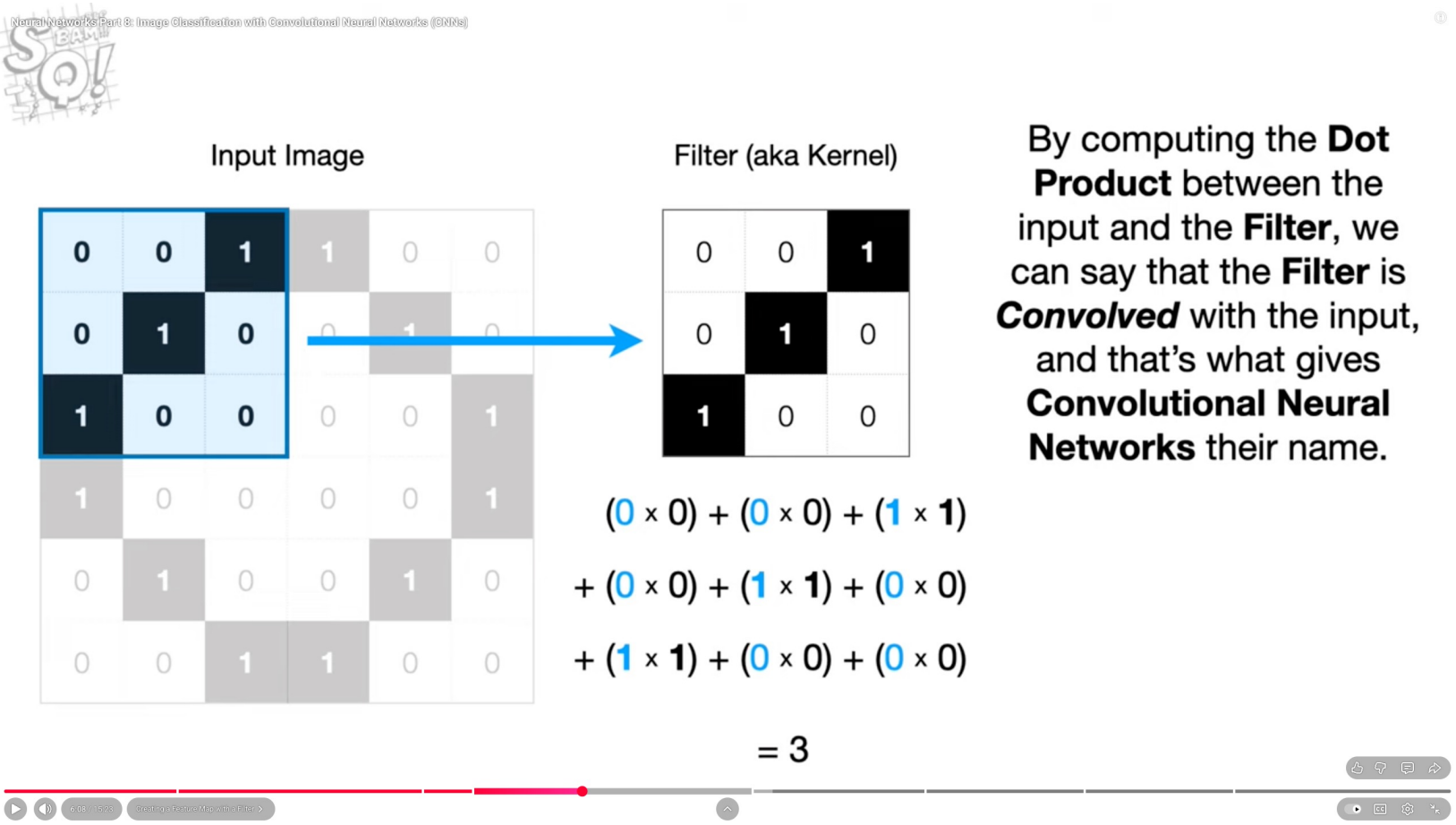
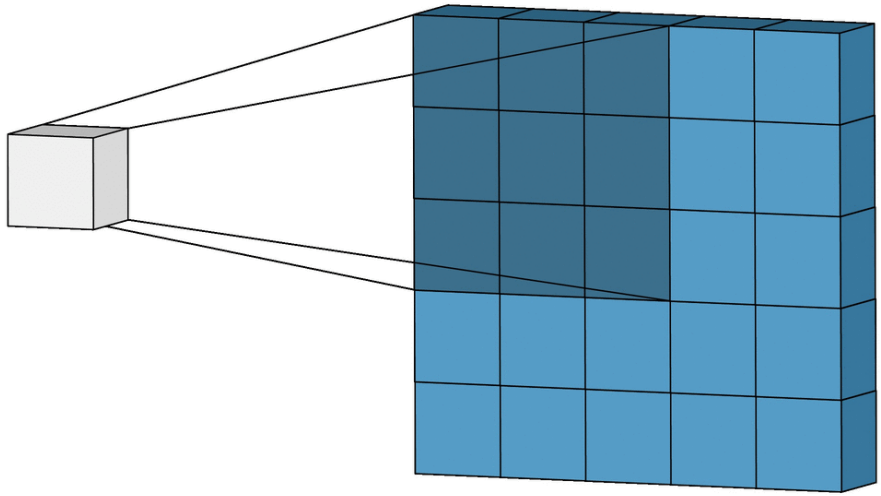


Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

Convolution Operation

The same weights are used at each position of the input image.



1. Apply a 3x3 kernel on a certain position of the image
2. Shift the position

Taken from the great website on convolution: https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1

Convolution Operation

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input X

1	0	1
0	1	0
1	0	1

Kernel W

4	3	4
2	4	3
2	3	4

Result Z

Convolution (let's ignore bias b):

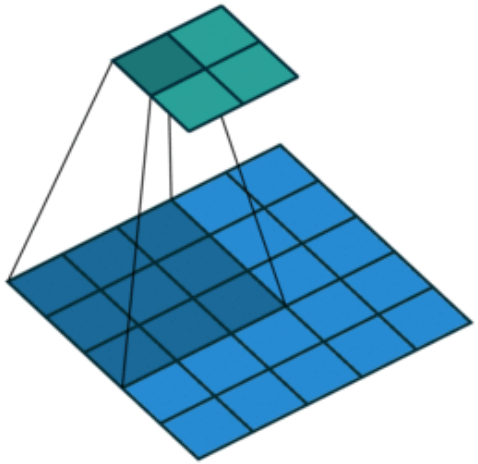
$$z = b + \sum_i x_i w_i$$

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

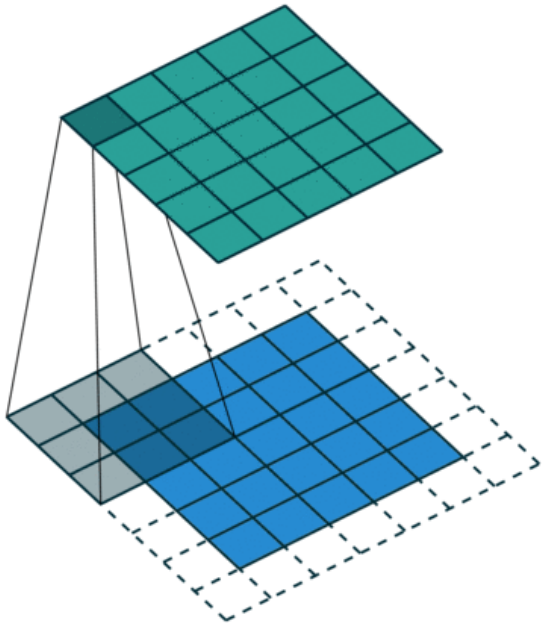
4		

https://medium.com/data-science/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2

CNNs use Zero Padding



no padding to only use
valid input information

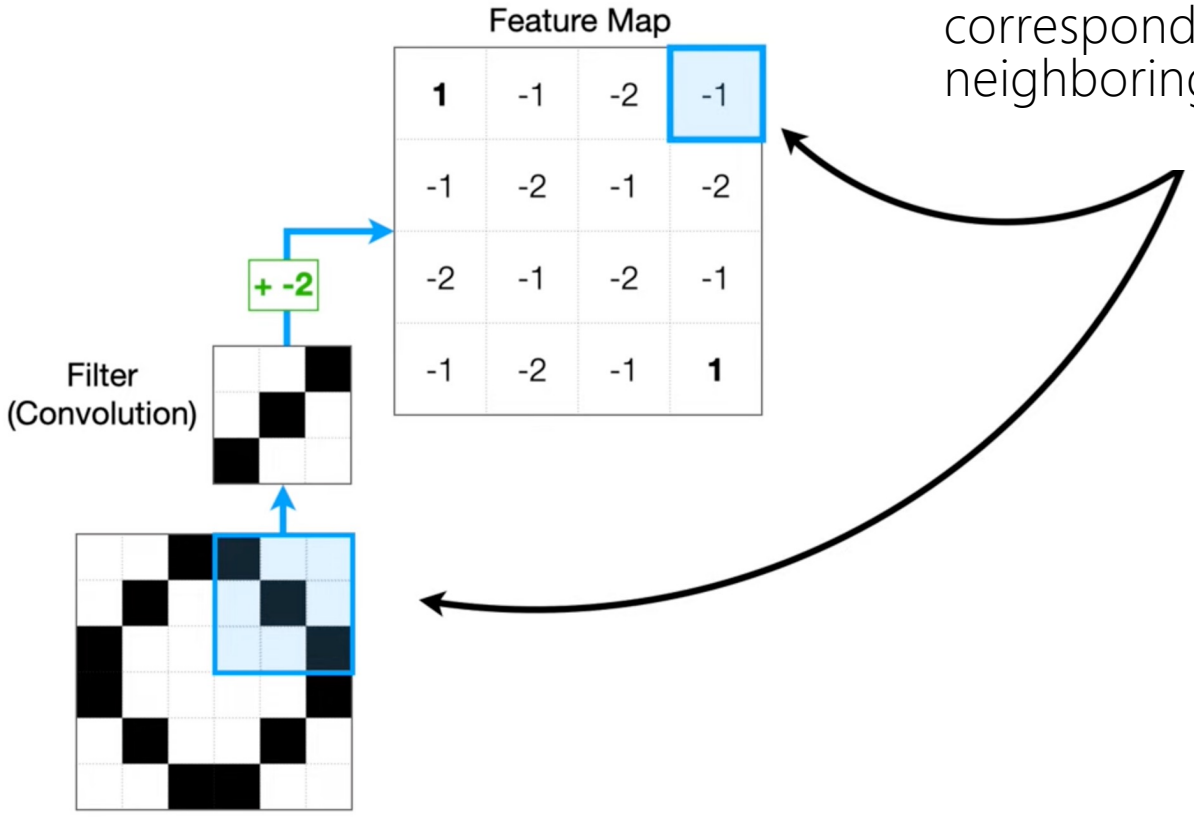


Zero-padding to achieve
same size of feature and input

The *same* weights are used at each position of the input image.

https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1

Convolving one filter on the entire image yields a full feature map.



Each cell in the feature map corresponds to a group of neighboring pixels

Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

CNNs use filters instead of single weights

Example: a 3x3 filter is a 3x3 matrix of weights.

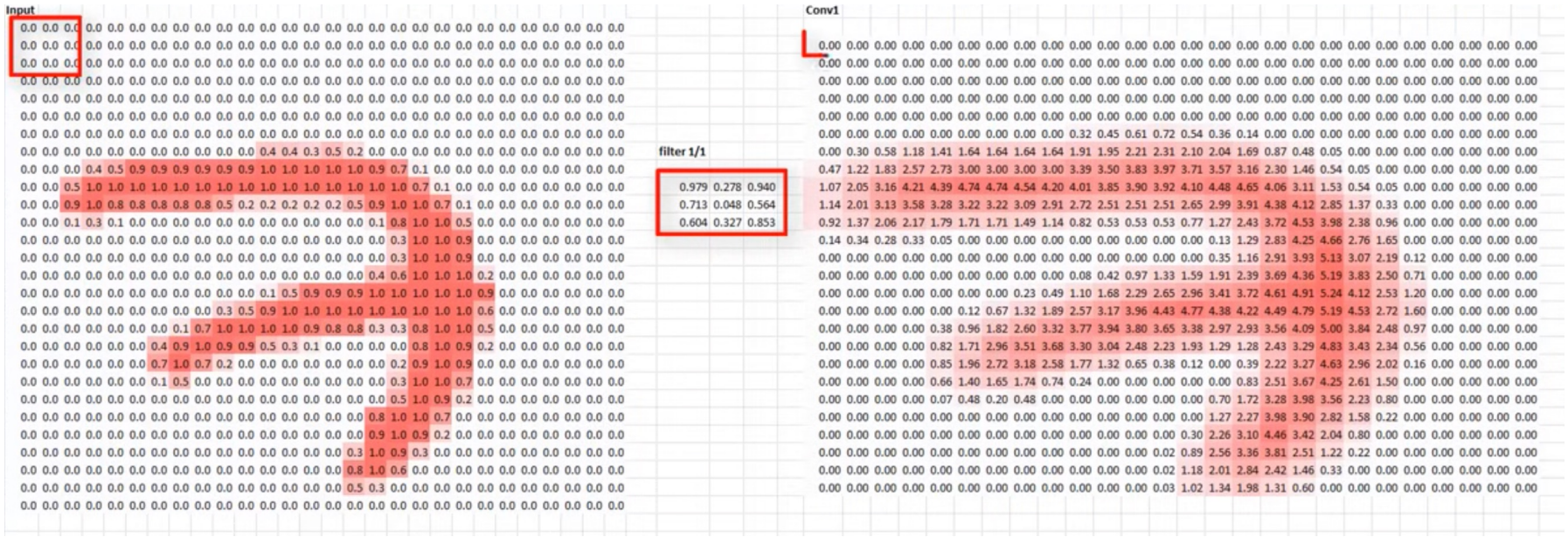
Initialize randomly

1. Calculate convolutions
2. Shift filter
3. Repeat

→Convolution output = feature map

Learn the filter weights by training with many input images

→final filters capture typical patterns like edges, corners, curves.



Convolutional Neural Networks (CNNs) explained https://www.youtube.com/watch?v=YRhxdVk_sIs

Different filters capture different patterns

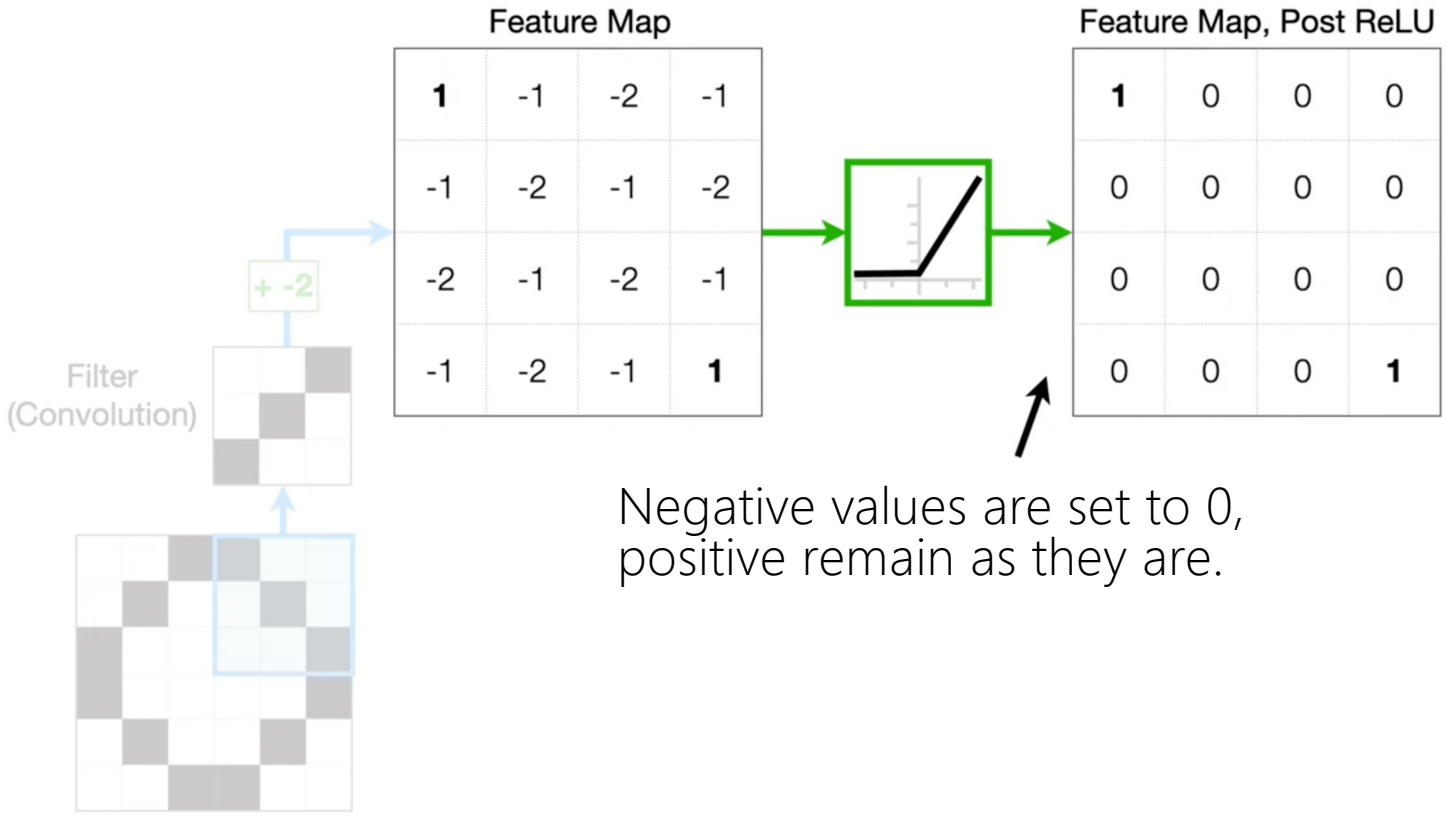




Convolutional Neural Networks (CNNs) explained https://www.youtube.com/watch?v=YRhxdVk_sIs

CNNs use Activation Functions (ReLU)

ReLU = Rectified Linear Unit



Negative values are set to 0, positive remain as they are.

Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

CNNs use Pooling

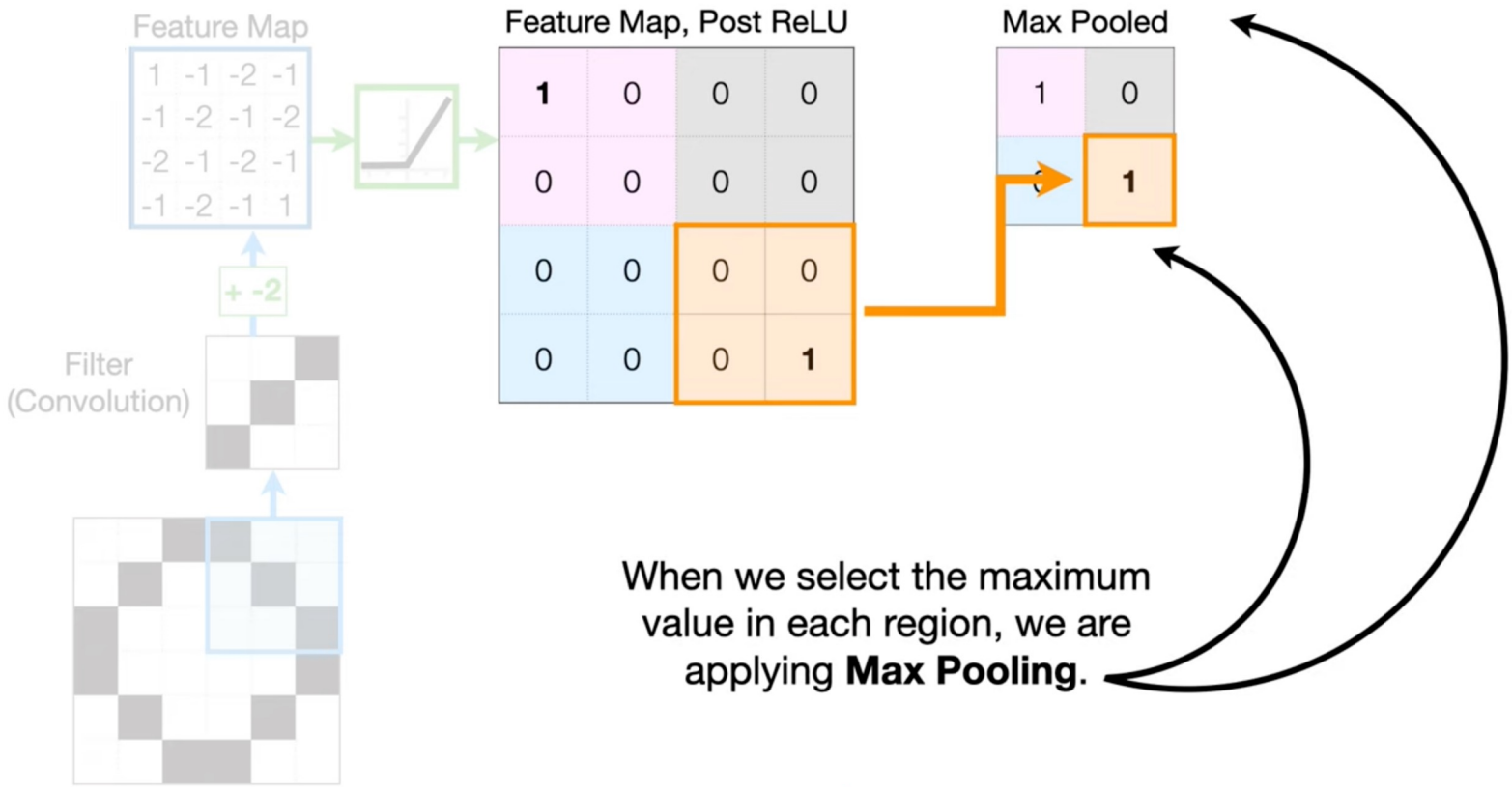
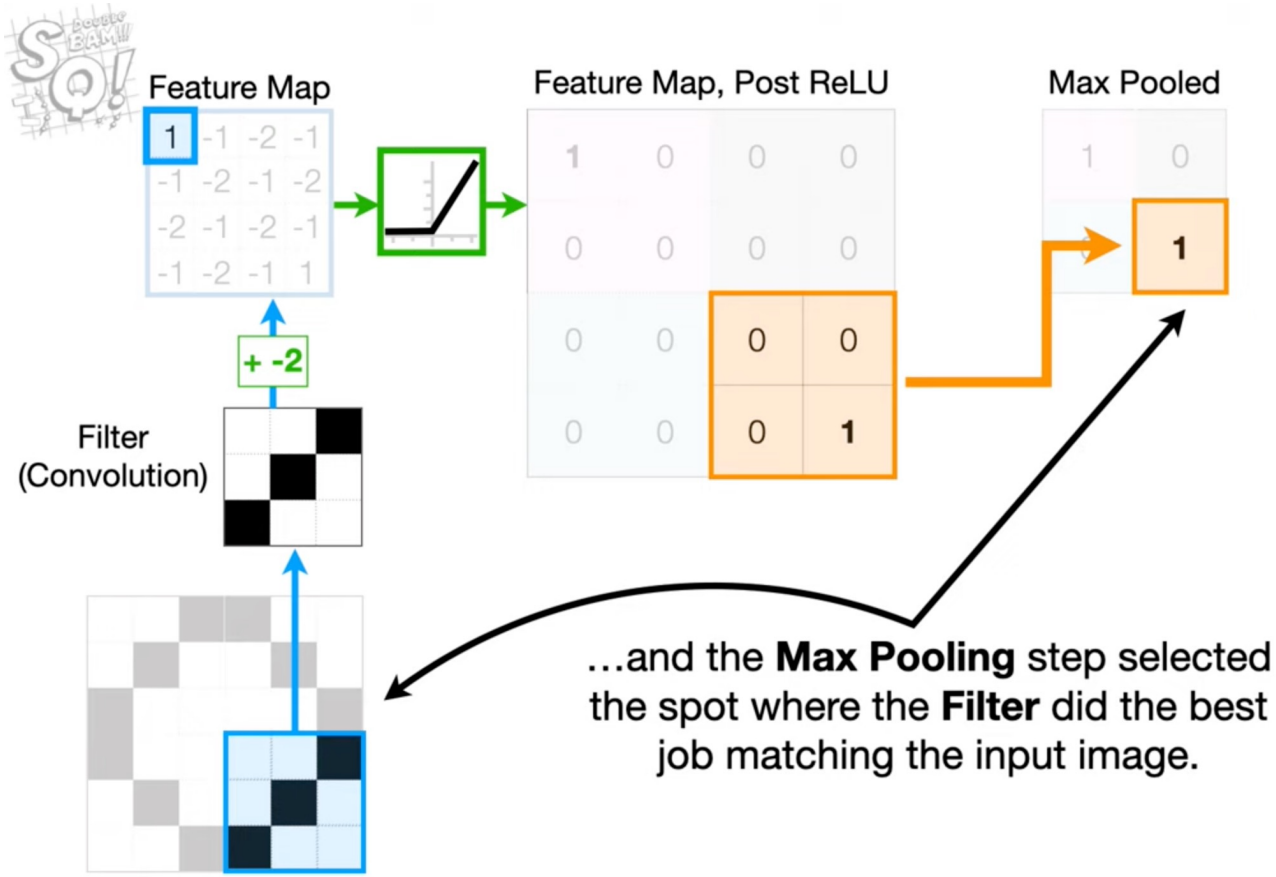
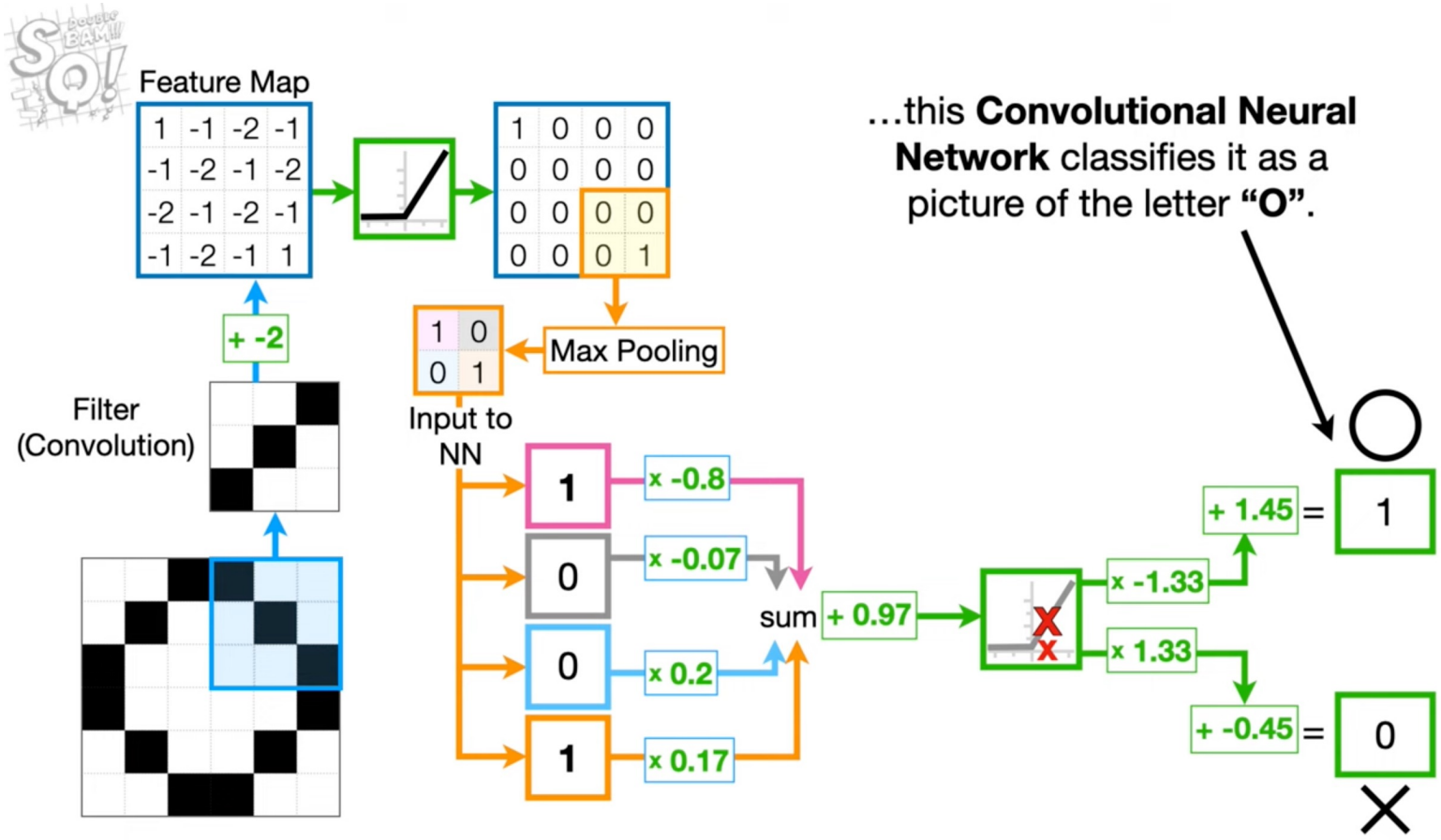


Image Classification with Convolutional Neural Networks (CNNs) https://www.youtube.com/@statquest

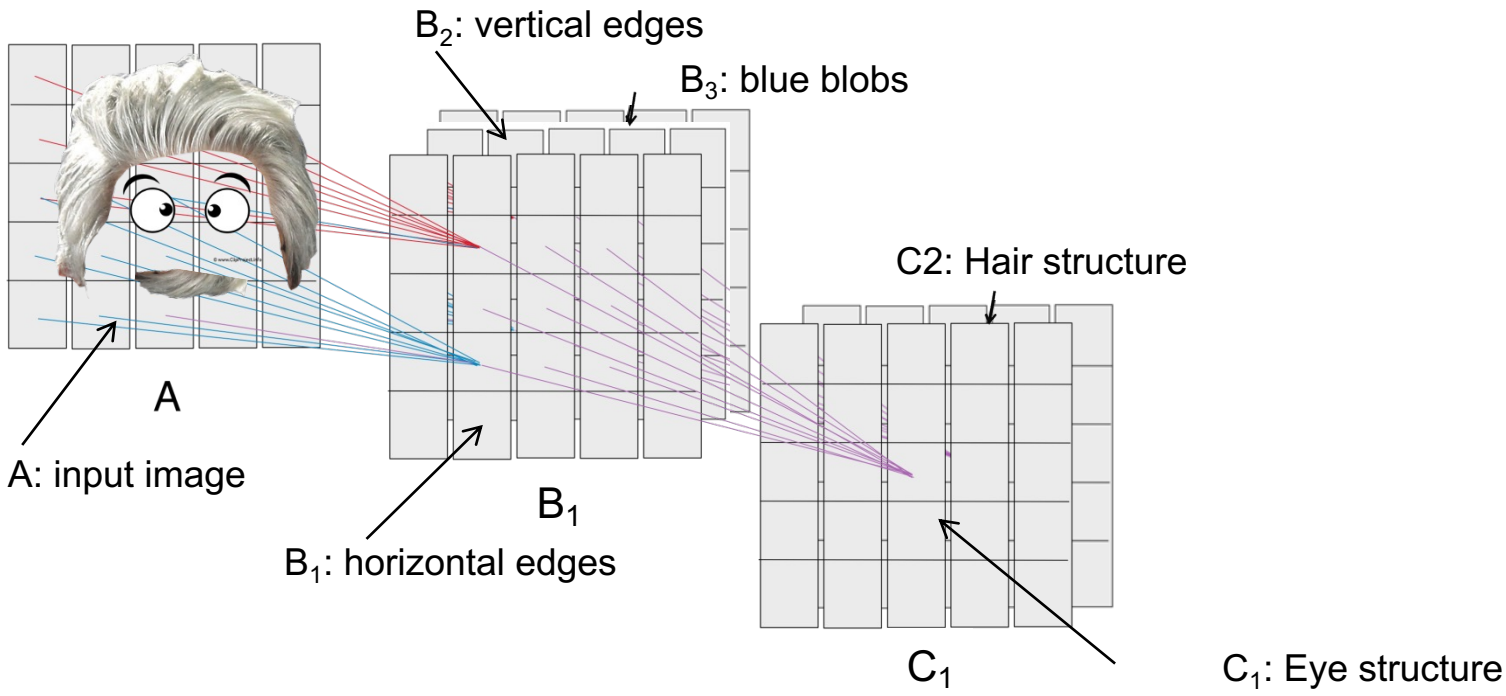
CNNs use Pooling



The rest is similar to FCNNs

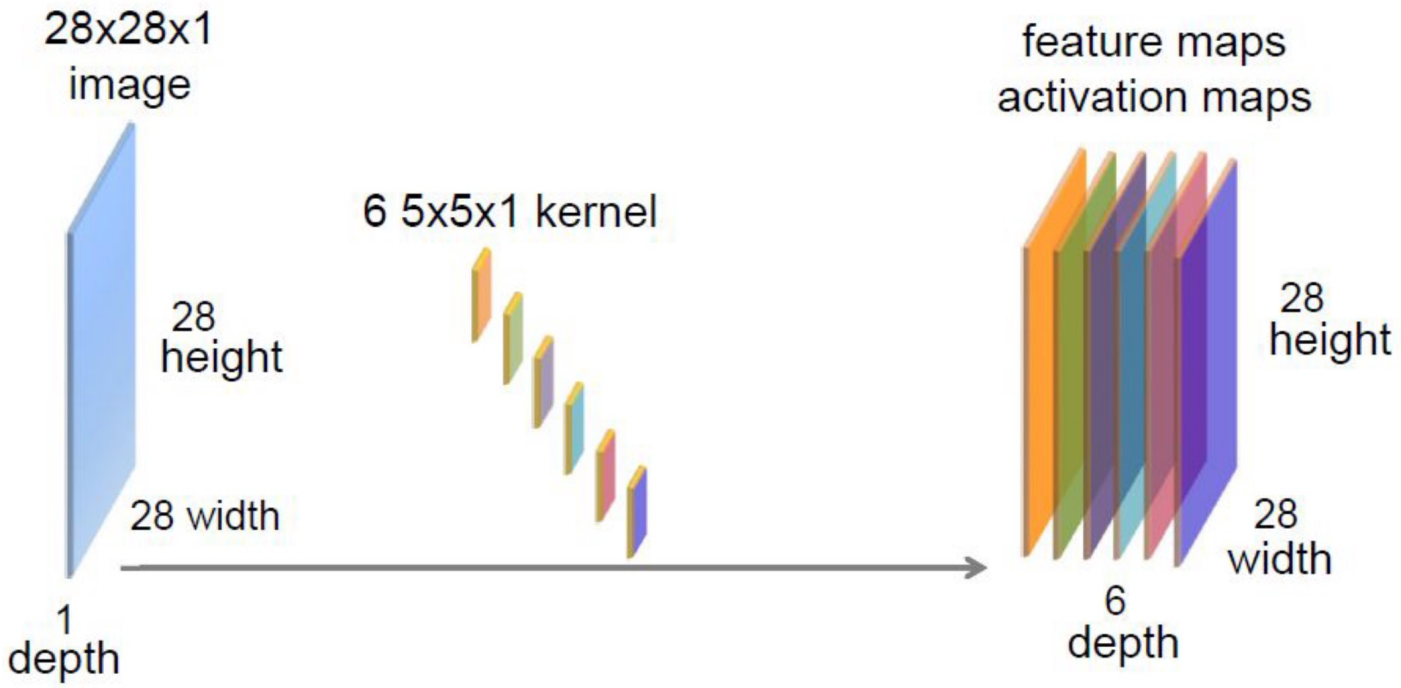


More than one kernel (motivation)

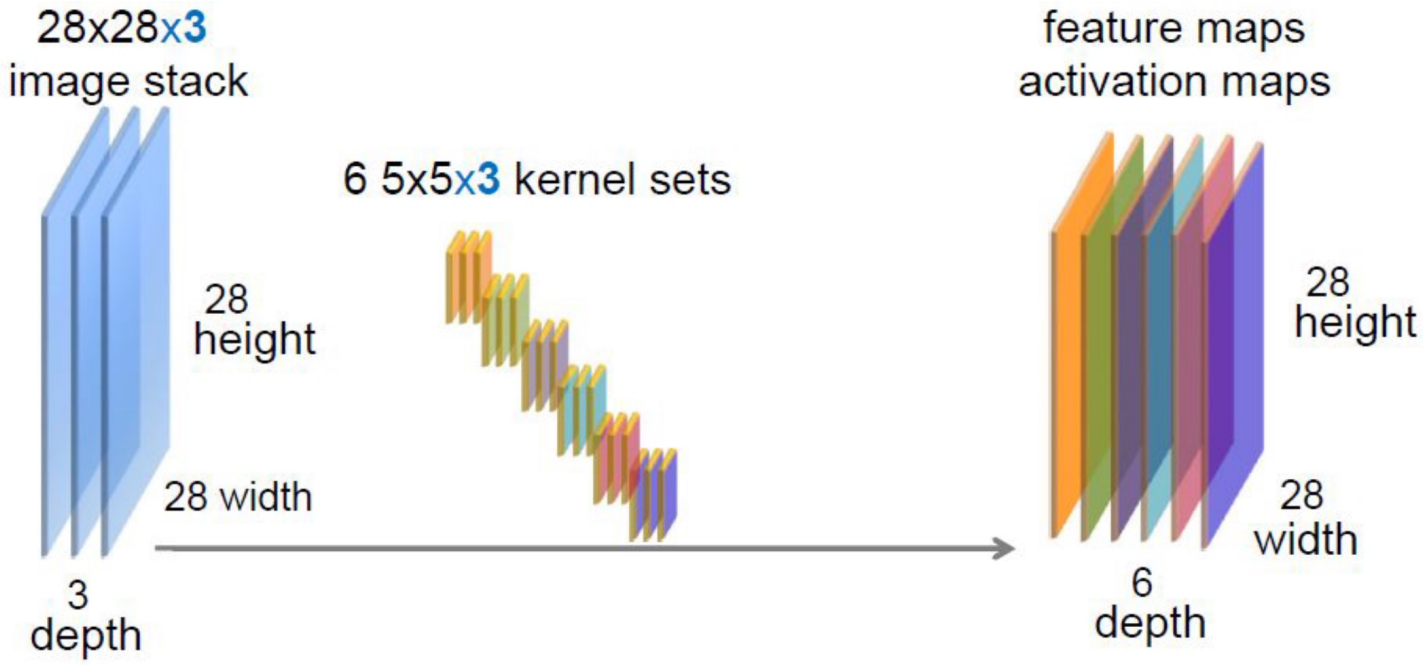


Multiple filters

Convolution of the input image with 6 different filters (kernels) results in 6 feature maps



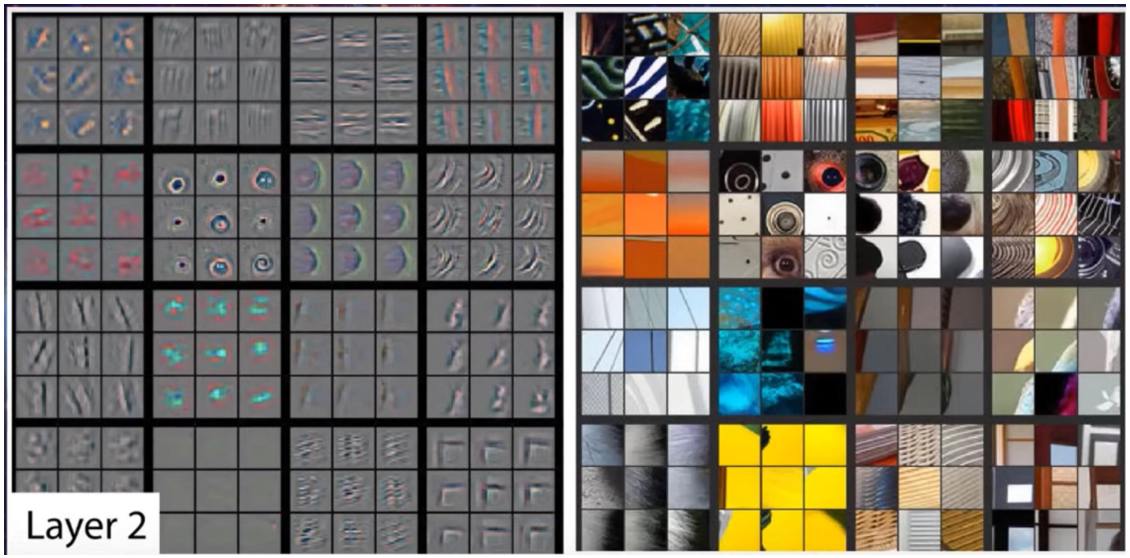
Convolution layer with a 3-chanel input and 6 kernels

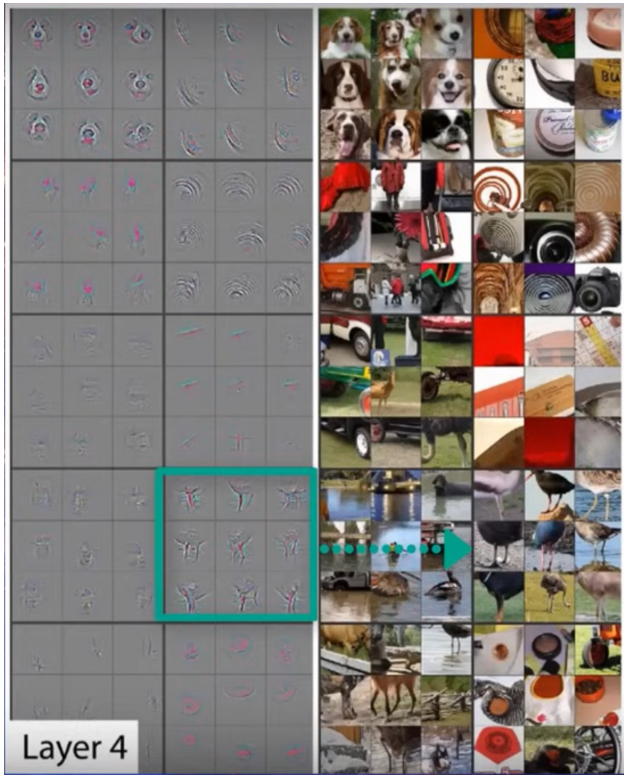


Convolution of the input image with 6 different kernels results in 6 activation maps.

If the input image has 3 channels, then each filter has also 3 channels.

Deeper Layers → Higher Level Features

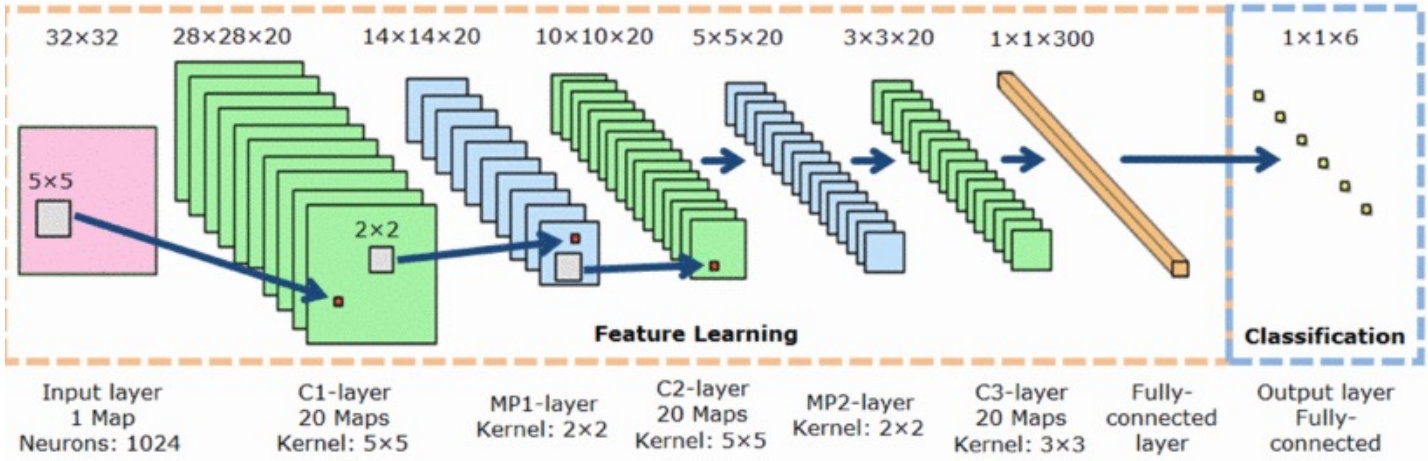




Receptive field

The receptive field of a neuron is the area in the original input image that impacts the value of this neuron = "that can be seen by this neuron".

Receptive field in original input: 5x5 6x6 14*14 16x16 24x24 32x32



Neurons from deeper layers have a larger receptive field than neurons closer to the input.

Code to determine size of receptive field: http://stackoverflow.com/questions/35582521/how-to-calculate-receptive-field-size

Feature extraction with pretrained foundation models

- Image classification is one of the most studied tasks.
- **Foundation models** trained with huge datasets extract informative features.
- Represent an image with its extracted features instead of raw pixels.

- Example: **ResNet152.**
  - 152-layer convolutional neural network.
  - Residual connections: helps efficient optimization (against vanishing gradients).
  - Trained on ImageNet dataset: 1.2 million images classified to 1000 classes.
  - Features extracted from the second-to-last layer ➔ 2048 dimensional feature vector.

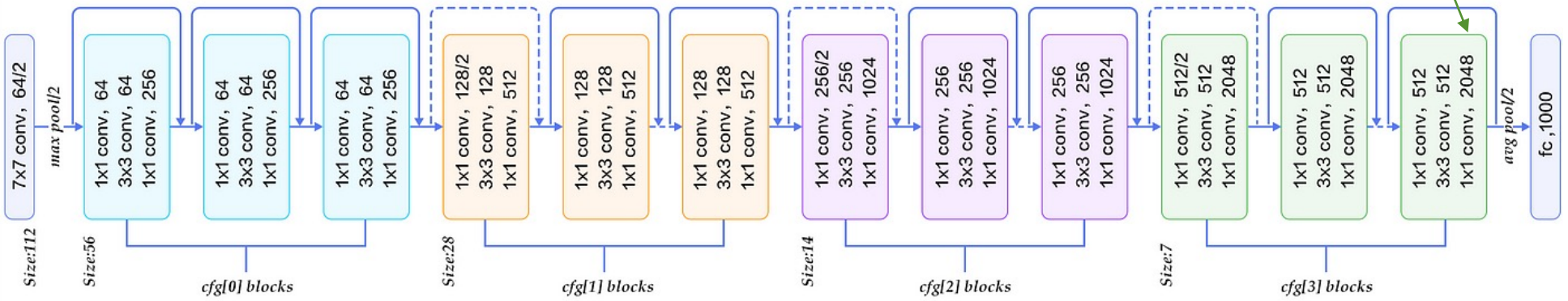
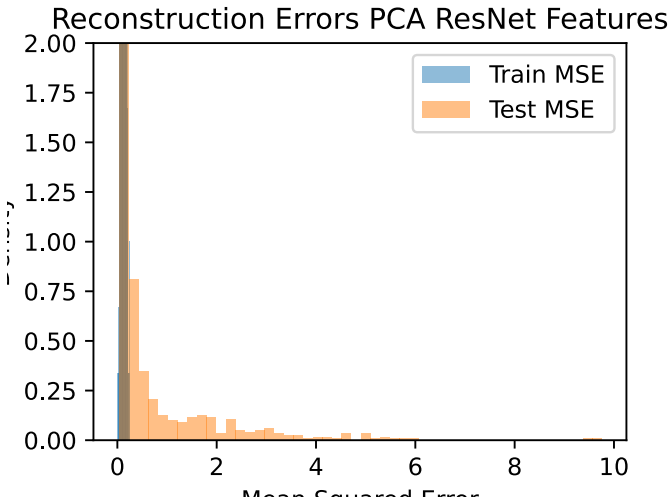


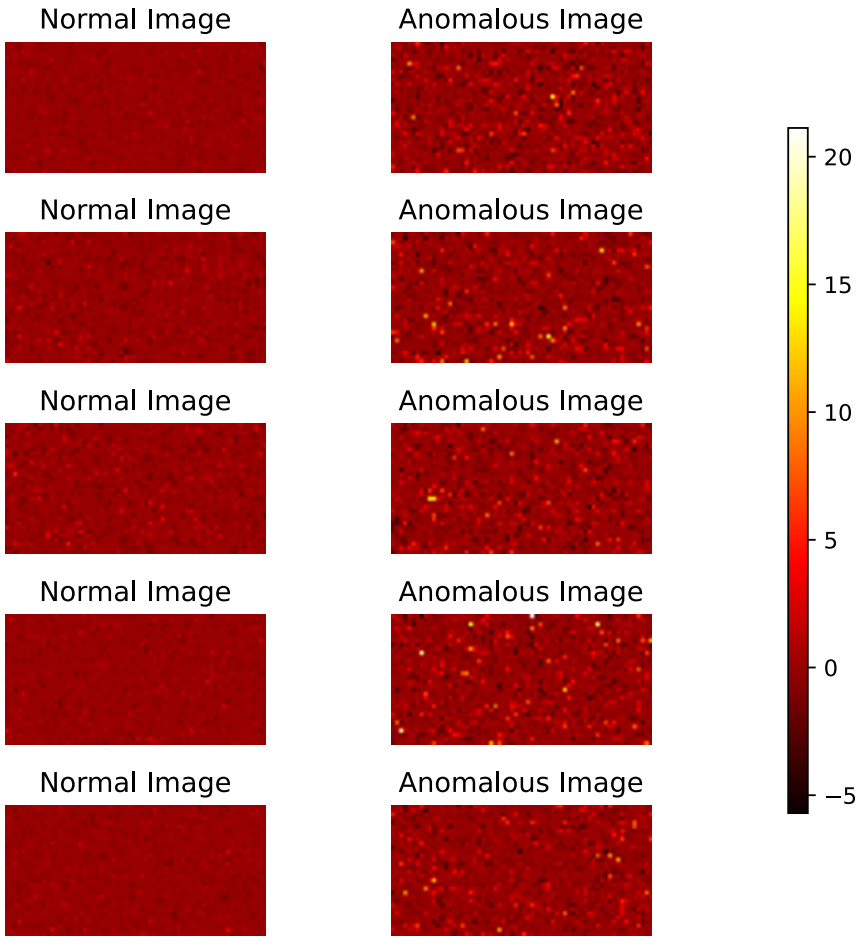
Image source: https://blog.devgenius.io/

PCA with ResNet152 Features

- Represent the feature maps as 2D images.
- Visualize the recon-error-map: original image – recon. Image
- Calculate the MSE per image:

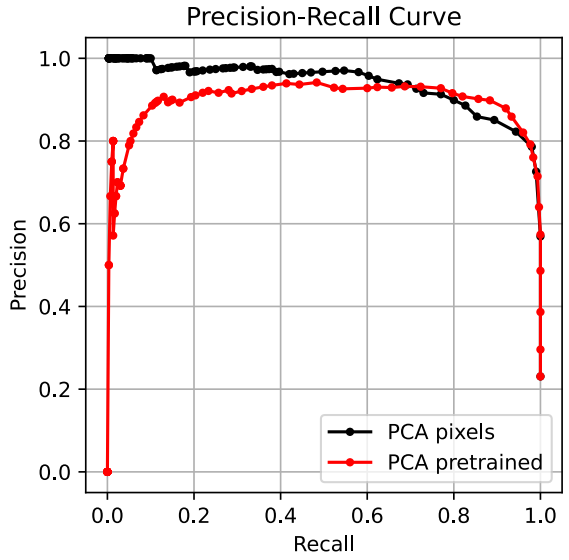


reconstruction-error-maps



Compare PCA models

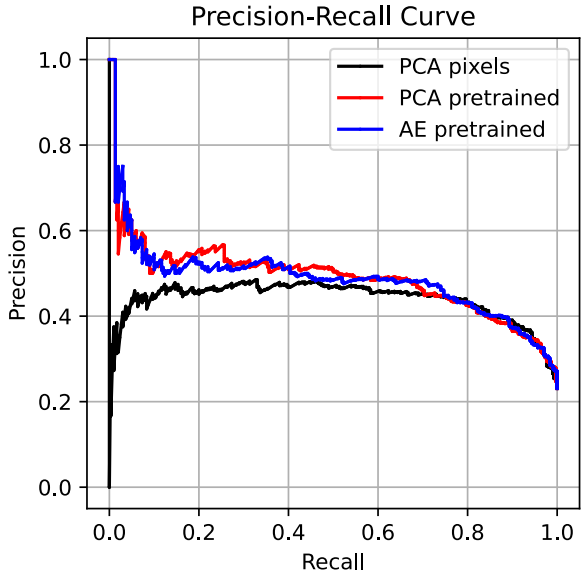
- Model 1: raw pixels as inputs
- Model 2: extracted features as inputs.
- Example: trousers vs. all.
- Advantage of pretrained features not always clear.



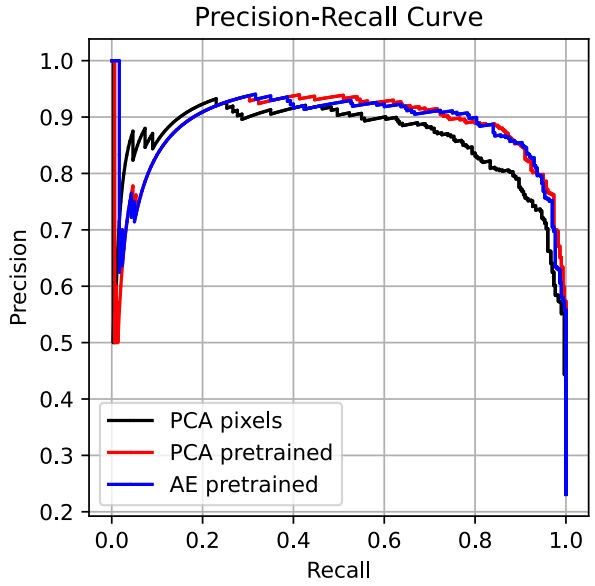
Reconstruction with PCA and AE

- Model 1: PCA raw pixels as inputs
- Model 2: PCA extracted features as inputs.
- Model 3: AE extracted features as inputs.
- For high precision and recall ➔ pretrained features are better.

Hard: T-shirt vs. dress, coat, shirt.

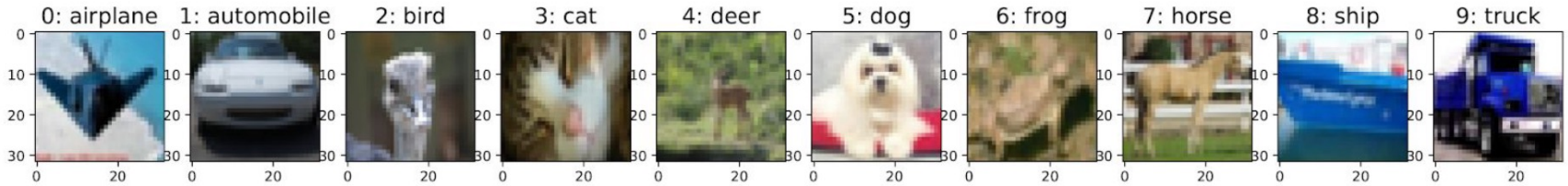


Easy: trousers vs. dress, coat, shirt.



- WPI_AnoDet_06a_PCA_AE_FMNIST_pretrained_feature_extractor
- WPI_AnoDet_06c_solution_PCA_AE_FMNIST_pretrained_feature_extractor_TSNE

CIFAR10 classification exercise



We define a CNN to classify cifar10 images (we have 10 classes)

Systematic model development

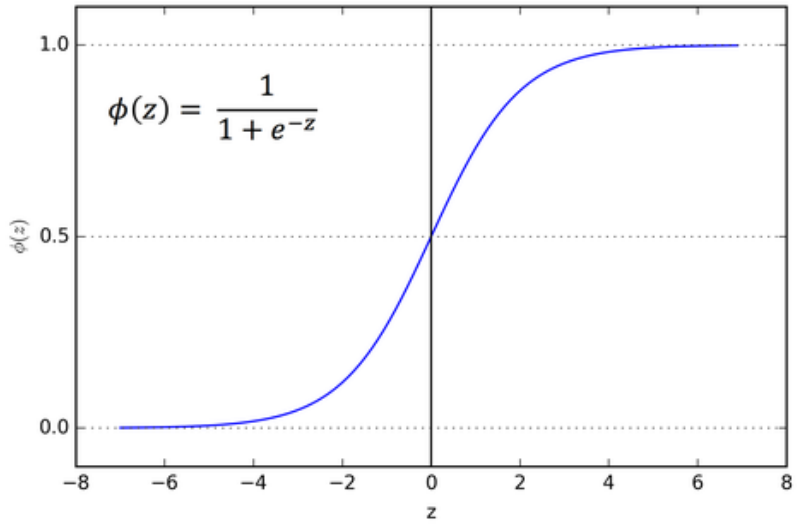
1. Choose a simple baseline model: e.g ML (random forest)
2. Define an initial CNN architecture

```
model = Sequential()


model.add(Conv2D(16, (3,3), activation="relu", padding="same",
          input_shape=(32,32,3)))
model.add(Conv2D(16, (3,3), activation="relu", padding="same"))
model.add(MaxPooling2D((2,2)))


model.add(Conv2D(32, (3,3), activation="relu", padding="same"))
model.add(Conv2D(32, (3,3), activation="relu", padding="same"))
model.add(MaxPooling2D((2,2)))


model.add(Flatten())
model.add(Dense(128))
model.add(Activation('relu'))
model.add(Dense(10))
model.add(Activation('softmax'))
```



3. Improve the model systematically.

Baseline architecture considerations

1. Shallow: CIFAR-10 images are tiny (32×32). No need for a very deep NN to start extracting useful patterns. keep small to train fast (not SOTA):
   - Stage 1 (16 filters) learns low-level features: edges, corners, simple patterns.
   - Stage 2 (32 filters) learns more complex combinations: parts of objects, textures, shapes.
2. Small 3×3 kernels are the standard building block (increases effective receptive field with fewer parameters).
3. Pooling reduces resolution and adds translation tolerance: MaxPooling2D((2,2)) halves spatial resolution:
   - 32×32 → 16×16 after first pool
   - 16×16 → 8×8 after second pool
4. Growing no. of filters. Modern CNNs follow this rule almost universally: As spatial resolution decreases, filter number increases:
   - At 32×32: No need for many filters to detect simple edges. 16 filters is enough for basic low-level features.
   - At 8×8: Each unit corresponds to a much larger part of the image. The network needs more filters (e.g 32) to encode different object parts and class-specific combinations.
5. A small dense head (128 → 10) limits memorization (overfitting).

WPI_AnoDet_06b_cifar10_tricks

Model improvement: tricks of the trade

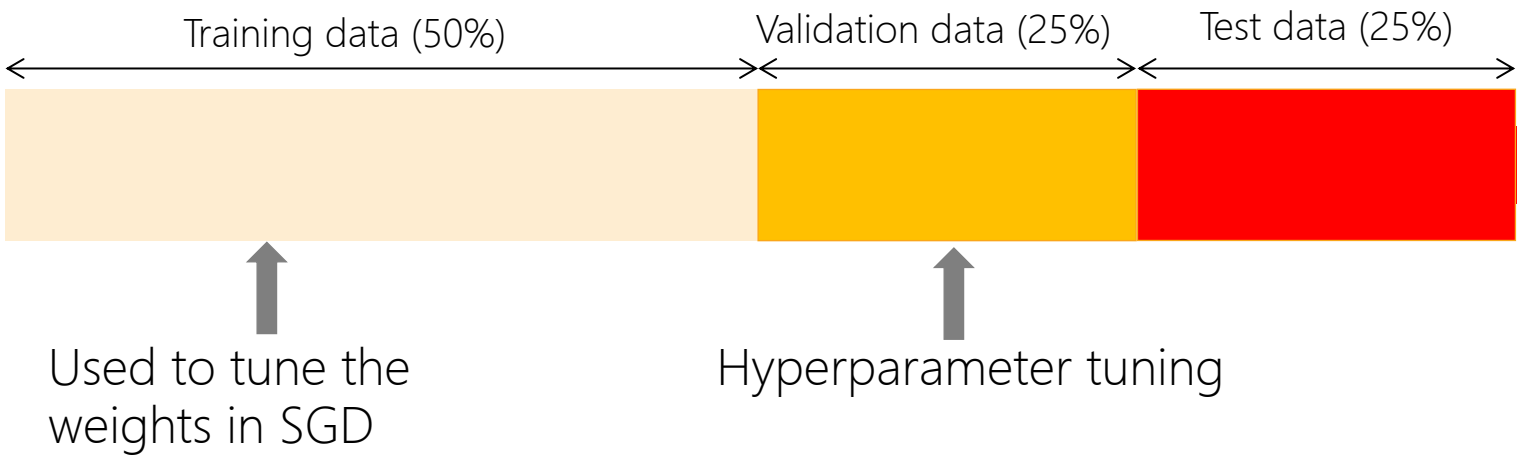
Some of the common practices that can improve performance:

1. Early stopping
2. Dropout
3. Data augmentation
4. Network capacity increase
5. Batch normalization
6. Weight decay

For **systematic model improvement**: important to understand their effects

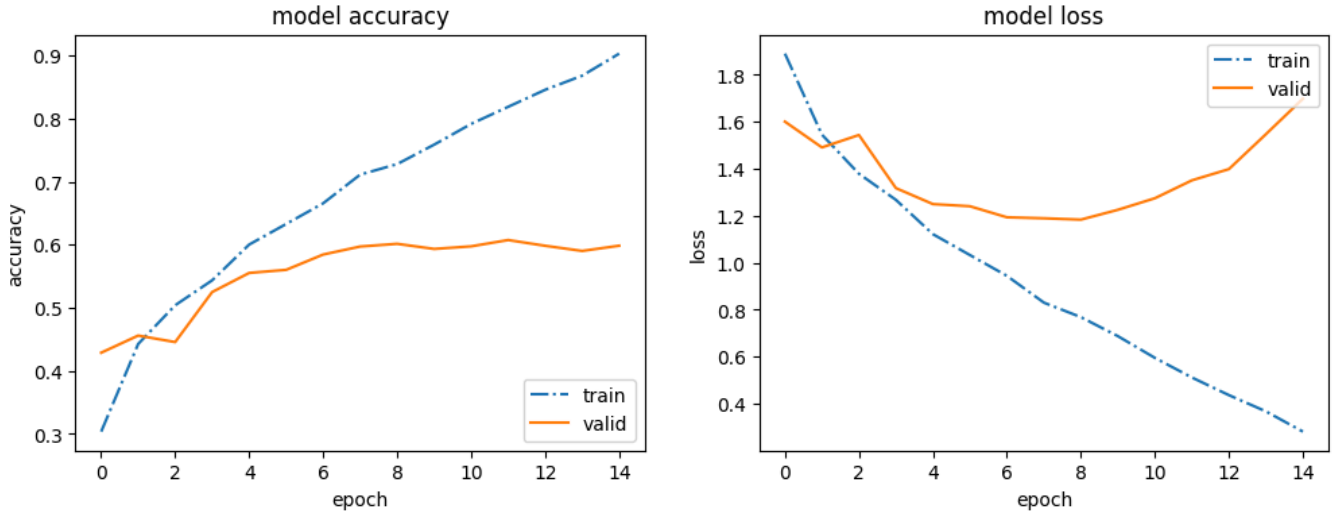
➔ Let us examine them one by one

Best practice: split to training, validation, and test set



Best practice:
Lock an extra test data set away, and use it only at the very end, to evaluate the chosen model, that performed best on your validation set.
Reason: When trying many models, you probably **overfit** on the validation set.

1. Early Stopping



Keep on training:
- Training loss keeps decreasing and training accuracy grows, but
- Validation loss starts growing (accuracy does not improve)

➔ CNN learns the training images by heart, without further optimizing features that can help classify new images.

➔ OVER FITTING!

➔ Stop training before the validation loss increases.

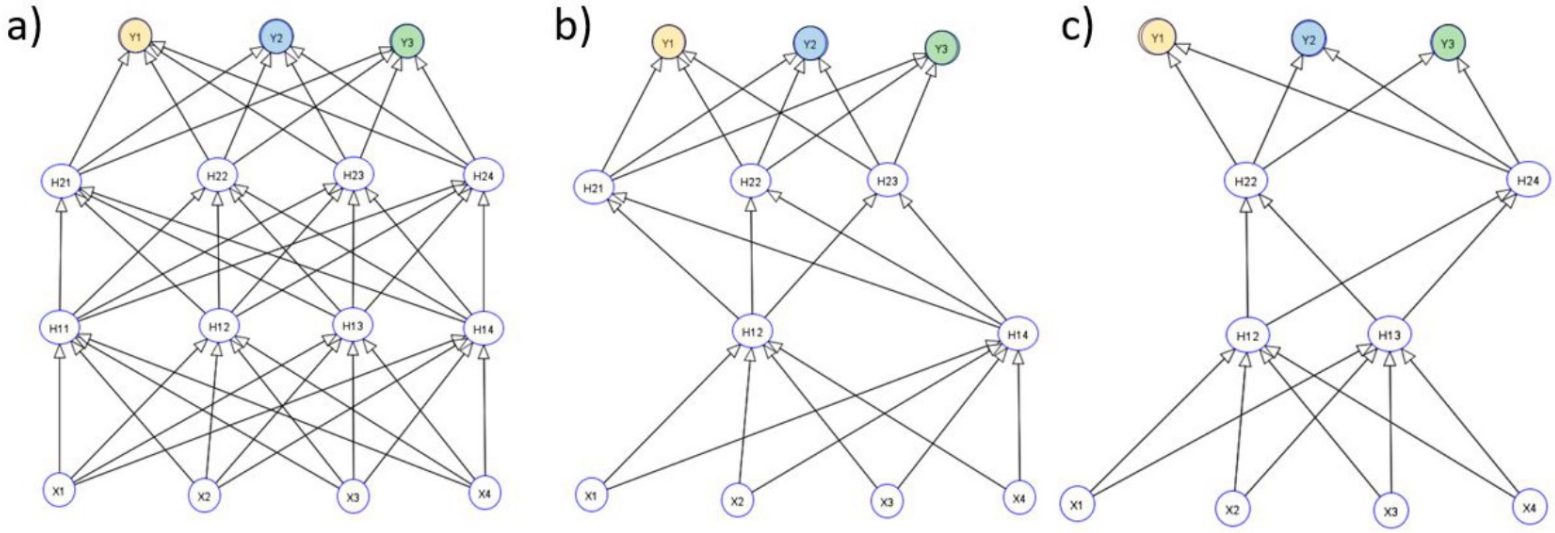
2. Dropout

A trick to prevent NN overfitting (not specific for CNNs)

- During training, we randomly "turn off" (drop out) a fraction p of all neurons in each step. p = dropout rate.
- Neuron is dropped in one batch ➔ no gradient ➔ no update
- Next batch ➔ if active ➔ updated normally

all neurons

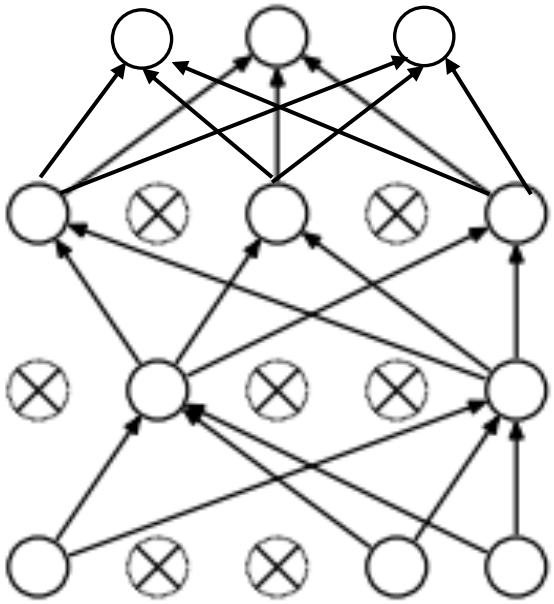
some connections are set to 0 = some neurons are dropped



Dropout forces the network to not put all its trust in a few neurons

➔ it has to spread knowledge around ➔ reduce memorization of specific data

How to implement dropout

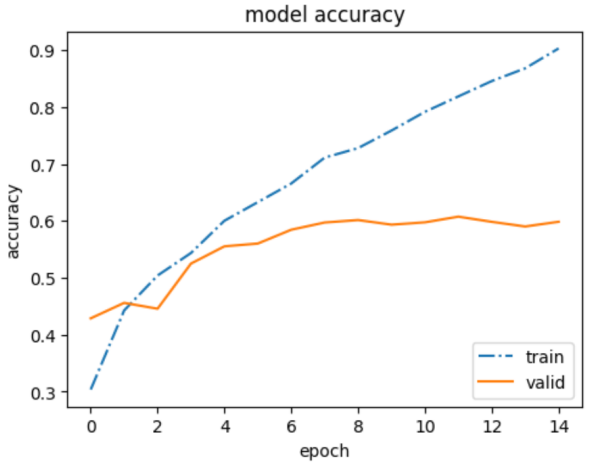


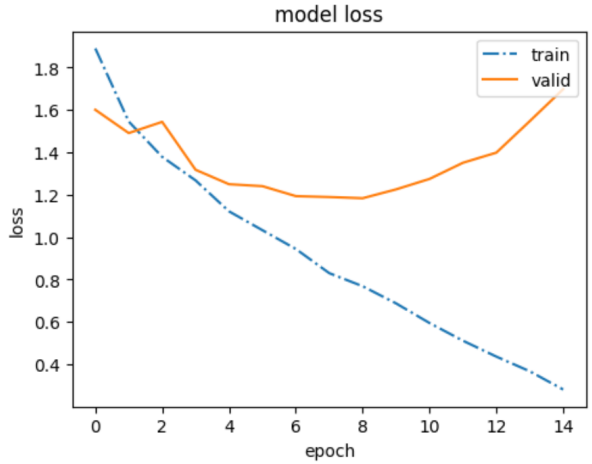
Using dropout during training implies:

- In each training step only weights to not-dropped units are updated → we optimize a sparse sub-model NN
- At inference no neurons are dropped.
- To get same expected output at training and inference: the inference weights are multiplied by the dropout probability p=0.5 (or the training weights divided by p).

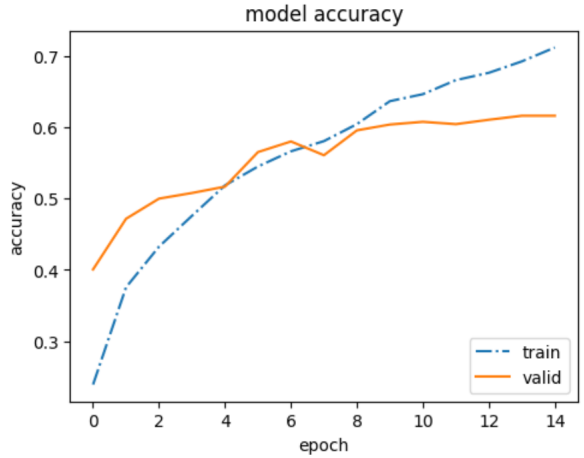
Effect of Dropout

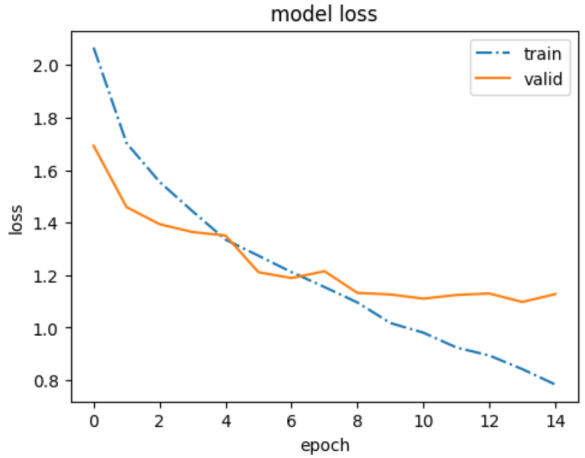
Without dropout





With dropout





	Acc
Random Forest (raw pixels)	0.3926
CNN (from scratch)	0.5995
CNN + dropout	0.6056

Dropout reduces overfitting and (slightly) increases the test accuracy

Further improvements

- dropout largely removed overfitting (still some left, can add more dropout layers).

- No big accuracy improvement on test data → the main classification difficulty is not due to overfitting but
  - the quality of the data and /or
  - the prediction capacity of the CNN model.

→ In the following we will improve generalization by

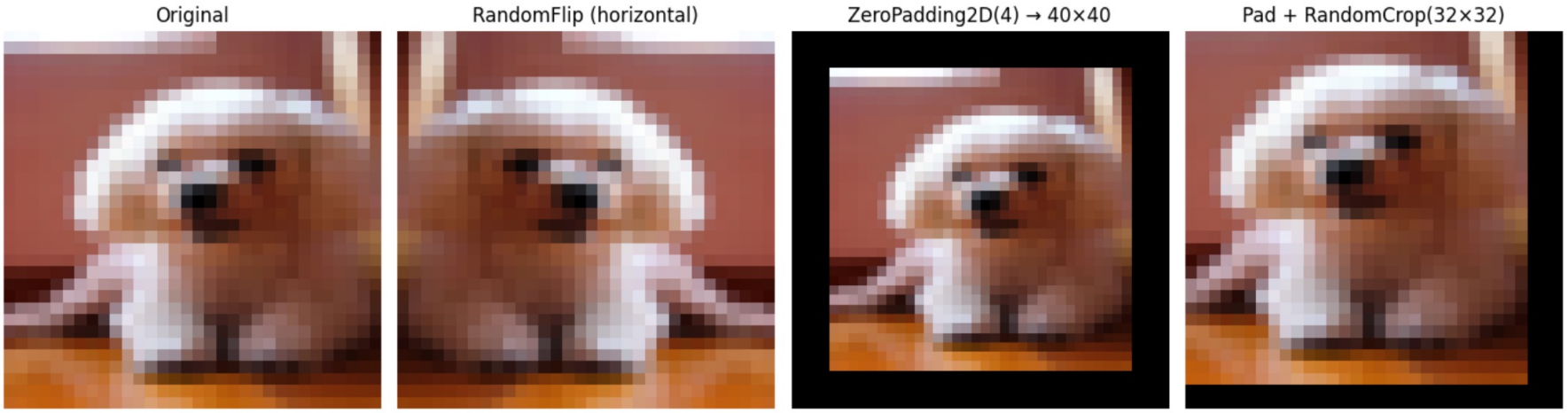
1) Augmenting the data.

2) increasing the model capacity.

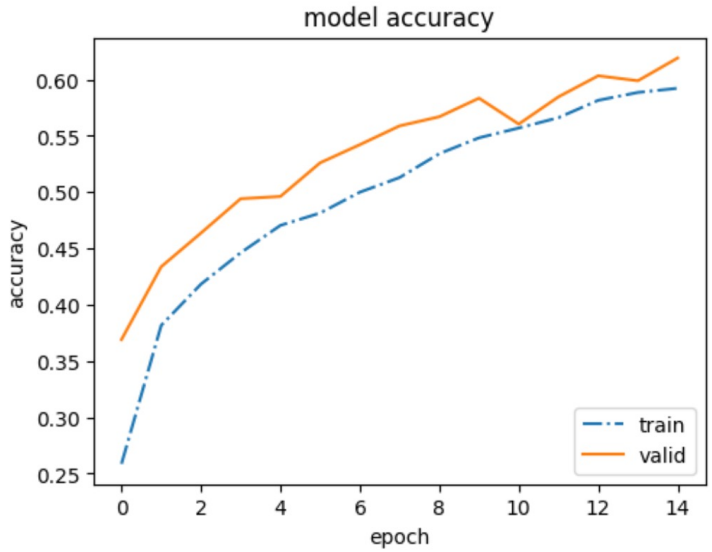
- Best practice: train with augmented data.
- Goal: make model invariant to orientation, rotation, flips, shifts, relative size… → help generalization.

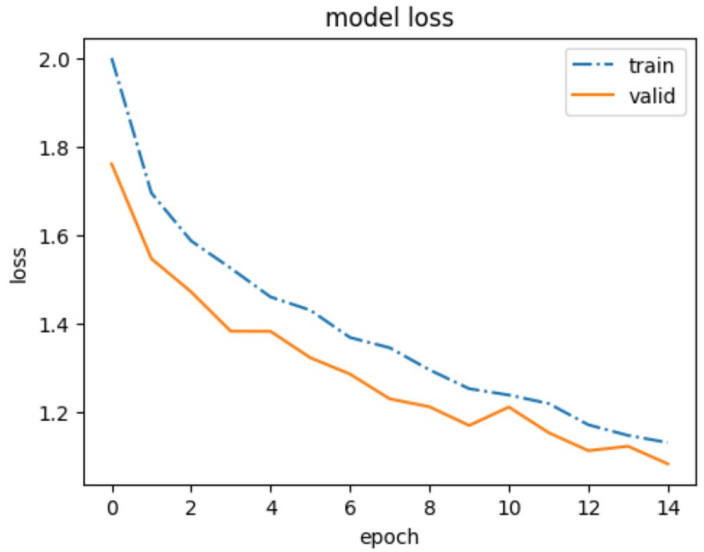
Examples:

- RandomFlip("horizontal"): randomly mirrors the image left↔right to make the model invariant to horizontal orientation changes.
- ZeroPadding2D(4): adds a 4-pixel border of zeros around the image so later random crops can shift the content without losing size.
- RandomCrop(32, 32): takes a random 32×32 window from a larger (padded) image to simulate small translations and framing changes.



Data augmentation effect






```
Augmented training model test accuracy: 0.6212000250816345
```

	Acc
Random Forest (raw pixels)	0.3926
CNN (from scratch)	0.5995
CNN + dropout	0.6056
CNN + augmentation	0.6212

Augmentation makes the training problem harder: The model sees shifted / flipped / cropped images ➔ Can no longer memorize exact pixel positions

➔ Training accuracy further decreases, this is expected (training on augmented data is harder).

➔ Data augmentation helps generalize: no overfitting thus higher test accuracy (test and valid. not augmented).

➔ Further improvements: increase the model's capability with more conv layers.

4. Increase CNN capacity

There are 2 main ways to increase capacity in a CNN:

- Go deeper (add more Conv layers)
- Go wider (increase number of filters per layer)

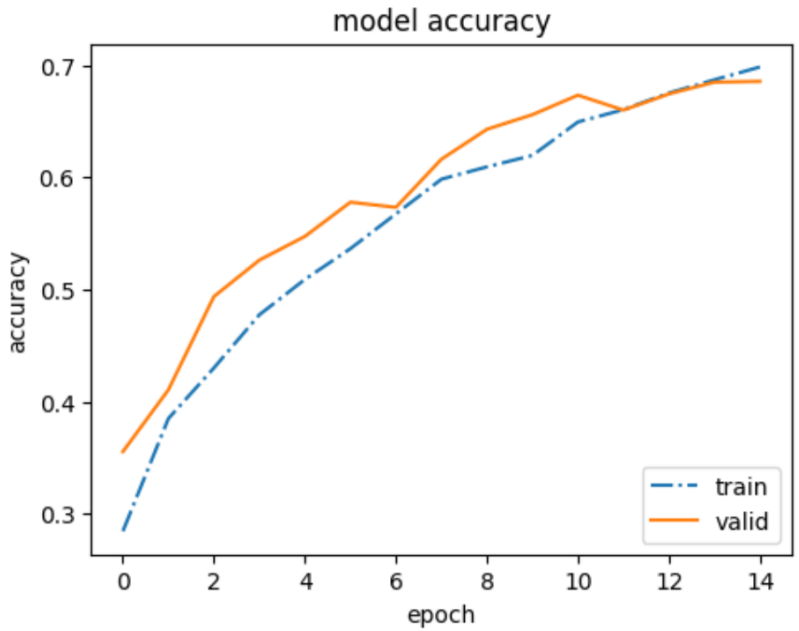
Both increase representational power, but they behave differently. Increasing the number of feature maps we

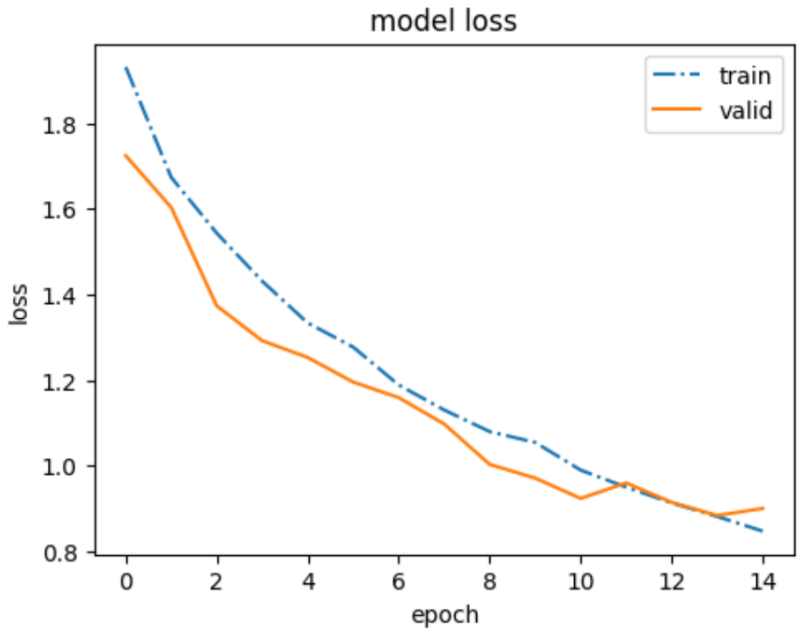
- let each spatial level learn more patterns
- Increase modelling capacity without changing gradient depth ➔ keep optimization stable

➔First step: go wider by doubling the number of filters per conv layer.

Trainable params: 280,218 ➔ 591,274 (2.26 MB)

Wider CNN: double the number of filters per layer






```
CNN (wider) + augmentation test accuracy: 0.6840000152587891
```

	Acc
Random Forest (raw pixels)	0.3926
CNN (from scratch)	0.5995
CNN + dropout	0.6056
CNN + augmentation	0.6212
CNN (wider) + augmentation	0.6840

WPI_AnoDet_06b_cifar10_tricks

Deeper CNN:

```
model_deeper3 = Sequential()

model_deeper3.add(Conv2D(16,(3,3),activation="relu",padding="same",
input_shape=(32,32,3)))
model_deeper3.add(Conv2D(16,(3,3),activation="relu",padding="same"))
model_deeper3.add(MaxPooling2D((2,2)))

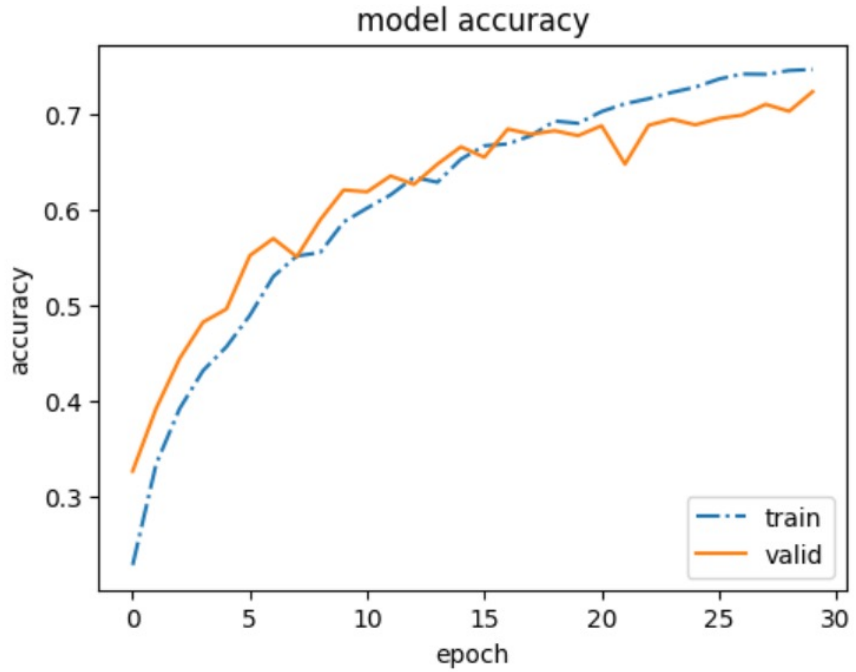
model_deeper3.add(Conv2D(32,(3,3),activation="relu",padding="same"))
model_deeper3.add(Conv2D(32,(3,3),activation="relu",padding="same"))
model_deeper3.add(MaxPooling2D((2,2)))

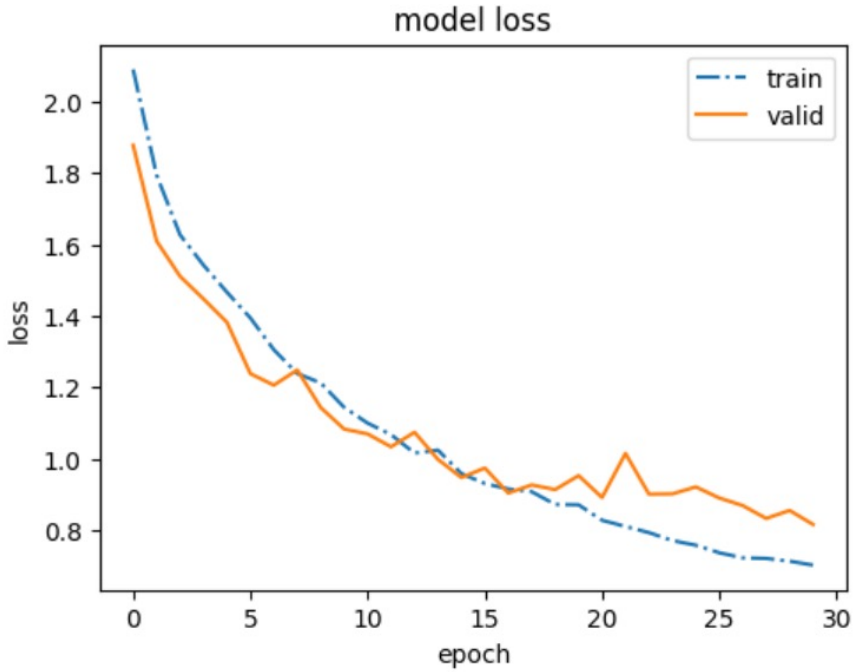
model_deeper3.add(Conv2D(64,(3,3),activation="relu",padding="same"))
model_deeper3.add(Conv2D(64,(3,3),activation="relu",padding="same"))
model_deeper3.add(MaxPooling2D((2,2)))
```

Deeper filters capture a bigger receptive field ➔ increase expressiveness by allowing for more filters as we go deeper.

Note: typically take longer to train!

Effect of Deeper CNN






```
Deeper CNN  + augmentation test accuracy: 0.7113000154495239
```

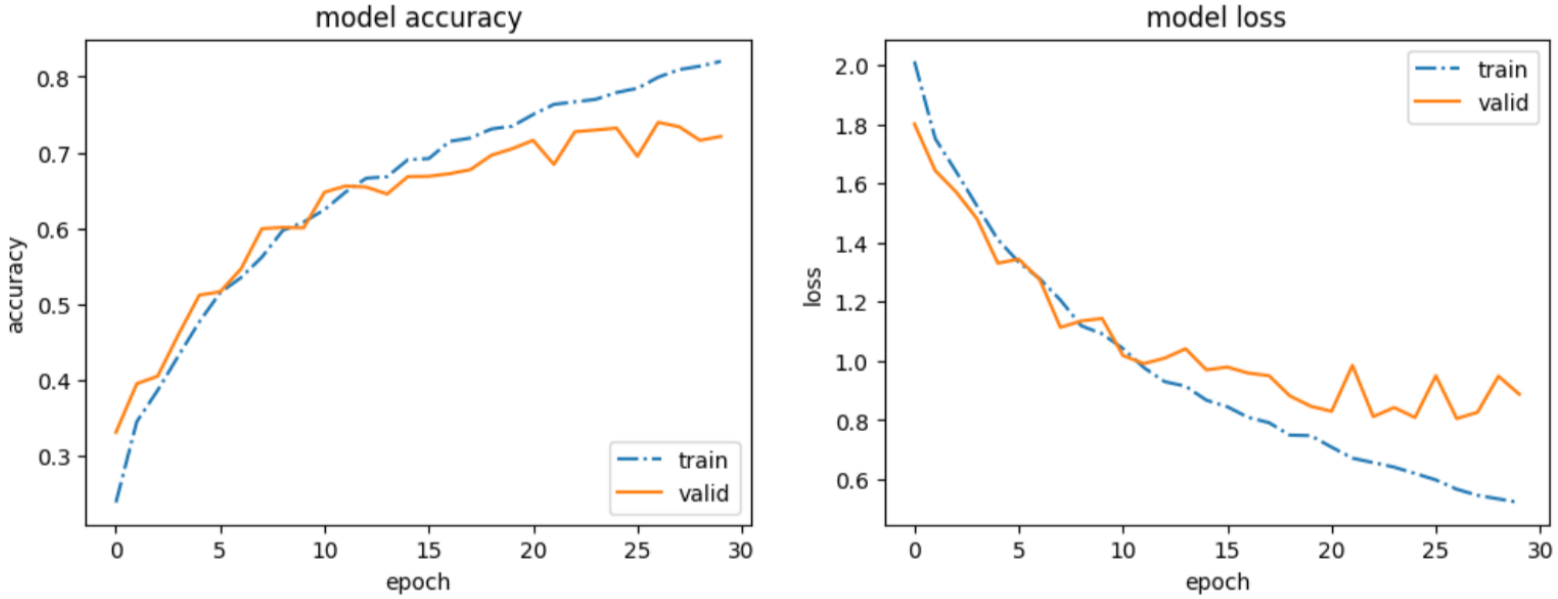
	Acc
Random Forest (raw pixels)	0.3926
CNN (from scratch)	0.5995
CNN + dropout	0.6056
CNN + augmentation	0.6212
CNN (wider) + augmentation	0.6840
CNN (deeper) + augmentation	0.7113

➔ Increase test accuracy with only slight overfitting (train acc: 0.75, val acc: 0.72)

Both deeper and wider?

Trainable params: 550,570

```
157/157 - 2s - 11ms/step - accuracy: 0.8201 - loss: 0.5210 - val_accuracy: 0.7211 - val_loss: 0.8874
```

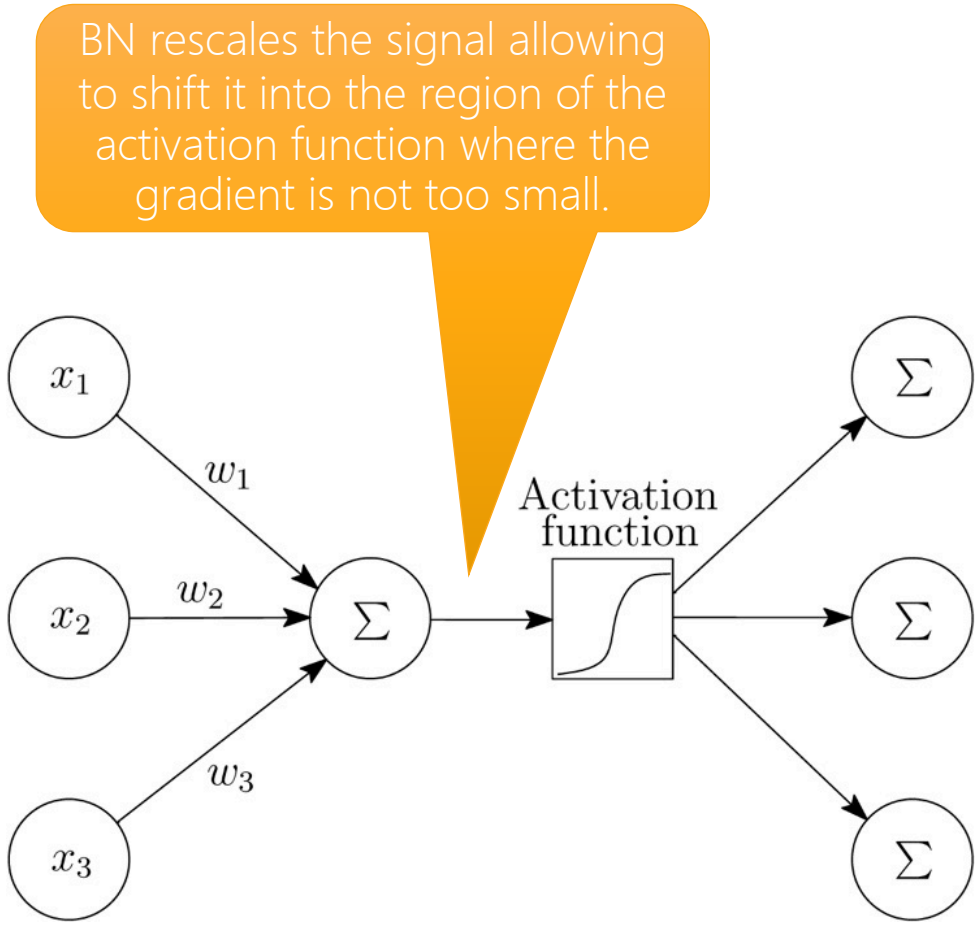


	Acc
Random Forest (raw pixels)	0.3926
CNN (from scratch)	0.5995
CNN + dropout	0.6056
CNN + augmentation	0.6212
CNN (wider) + augmentation	0.6840
CNN (deeper) + augmentation	0.7113
CNN deeper & wider + augmentation	0.7042

- ➔ Sightly decreased test accuracy but increased train acc
- ➔ Significant overfitting (train acc: 0.82, val acc: 0.72).
- ➔ Can we push test acc further by regularization?

Re-normalize the outputs of each layer after each batch

→ learn more effectively.



What is the idea of Batch-Normalization (BN)

Step 1: standardization.
Given outputs (activations) $\mathbf{x}$ from a layer, BatchNorm computes μ and σ of the batch and standardize the activations using: $\hat{x} = \frac{x-\mu}{\sigma}$

➔ A forced standardization with 0 mean and variance 1.

Step 2: allow a more flexible normalization.

- Add two learned parameters γ and β
- Learn them along with the weights
- Rescale the layer outputs with the learned parameters

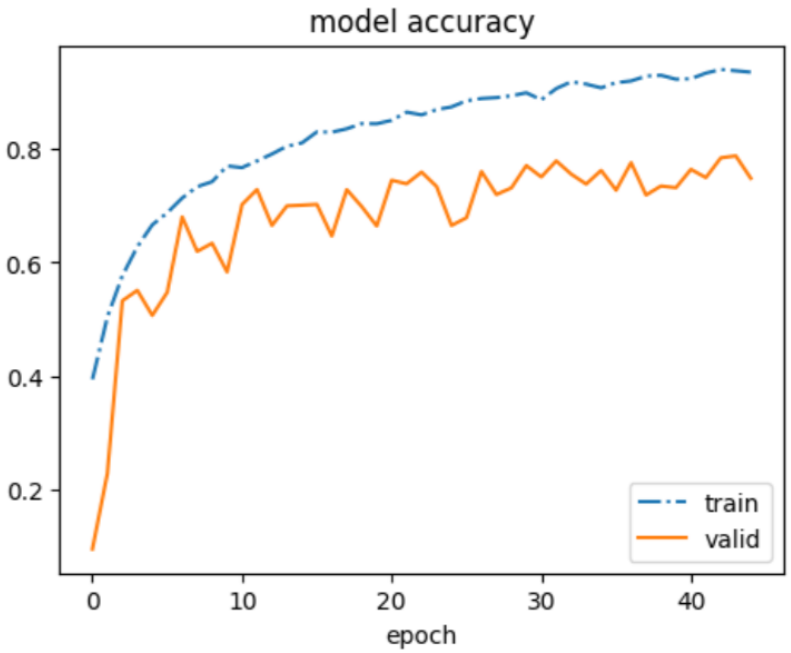
$$y = \gamma\hat{x} + \beta$$

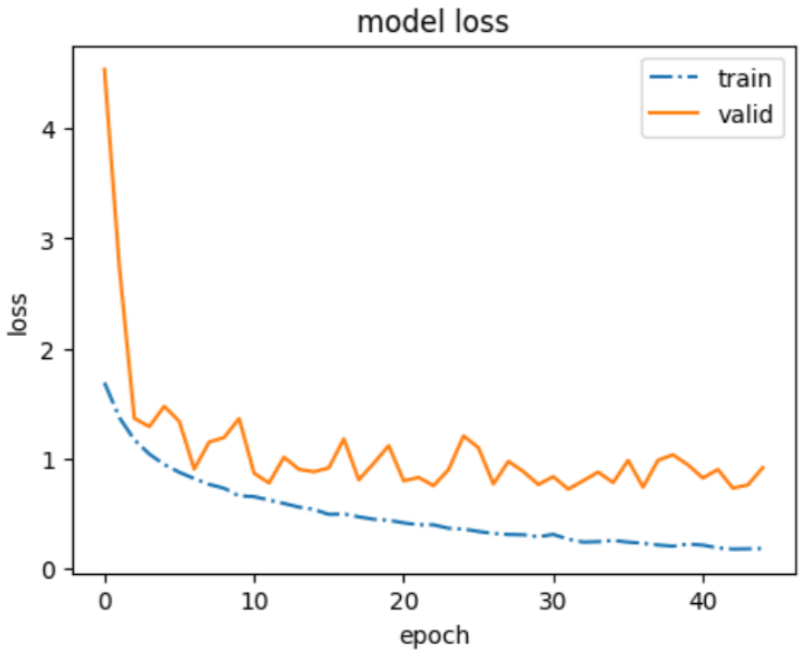
➔ This ensures we don't restrict what the layer can represent.

We achieve:

1. Normalized outputs enable effective optimization
2. Learnable normalization allows for flexible representation.

Effect of Batch Normalization





	Acc
CNN (from scratch)	0.5995
CNN + augmentation	0.6212
CNN (wider) + augmentation	0.6840
CNN (deeper) + augmentation	0.7113
CNN deeper & wider + augmentation	0.7042
CNN deeper & wider + BN + augmentation	0.7401

- ➔ Increased test accuracy but also train acc
- ➔ Significant overfitting (train acc: 0.93, val acc: 0.74).
- ➔ Can we push test acc further by more regularization?

- Weight decay is a technique that discourages the model from using very large weights.
- During training, instead of minimizing only the data loss Loss_{data} we minimize:

$$\text{Loss}_{data} + \lambda \sum W^2$$

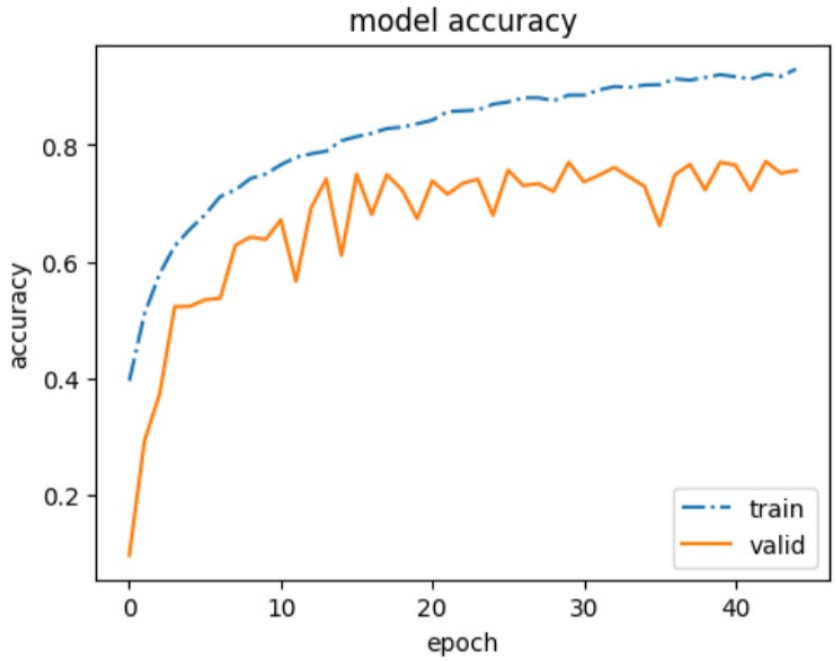
controls how strong the penalty is

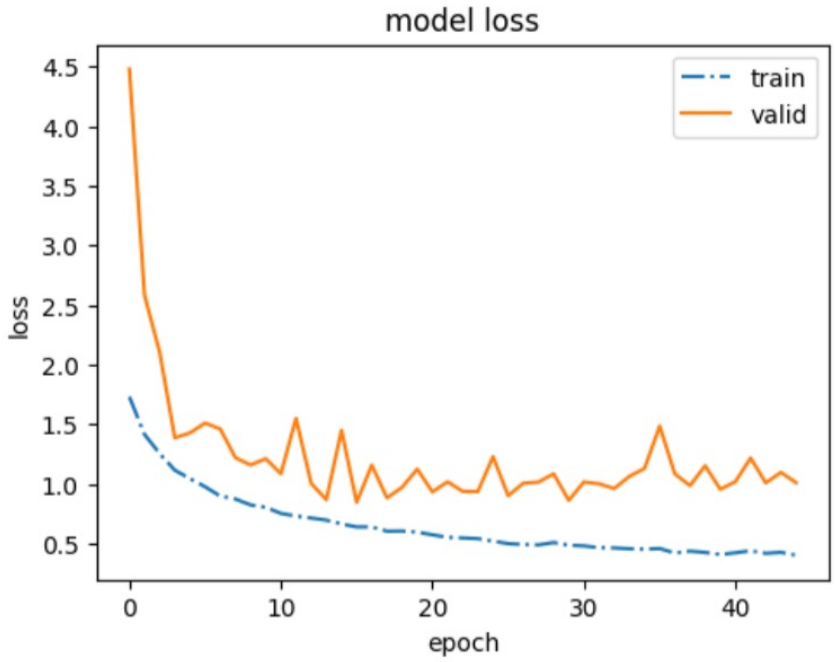
➔ add a small penalty proportional to the square of the weights.
➔ Large weights increase the loss.
➔The optimizer therefore prefers smaller weights
➔Weight shrink (decay) during training.

Intuition: two images that differ slightly should have similar outputs.
- Without weight decay the model may react strongly to tiny pixel variations.
- With weight decay the model becomes less sensitive.

➔ Nearby images map to similar outputs.
➔ Generalization improves ➔ less overfitting.

Effect of Weight Decay






```
Deeper CNN  + BN + augmentation + weight decay test accuracy: 0.7587000131607056
```

	Acc
CNN (from scratch)	0.5995
CNN + augmentation	0.6212
CNN (wider) + augmentation	0.6840
CNN (deeper) + augmentation	0.7113
CNN deeper & wider + augmentation	0.7042
CNN deeper & wider + BN + augmentation	0.7401
CNN deeper + BN + aug + wd=0.0001	0.7587

➔ Test accuracy may increase.

➔ Overfitting is reduced.

WPI_AnoDet_06b_cifar10_tricks

You cannot skip error analysis

Above we looked at the loss function and global accuracy to improve the model performance.

This **must** be complemented by an error analysis (local scores):

- Inspect wrong high anomaly scores
- Inspect wrong low anomaly scores

→ Understand where the model fails

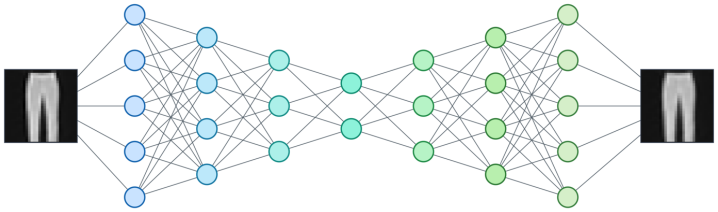
→ Improve

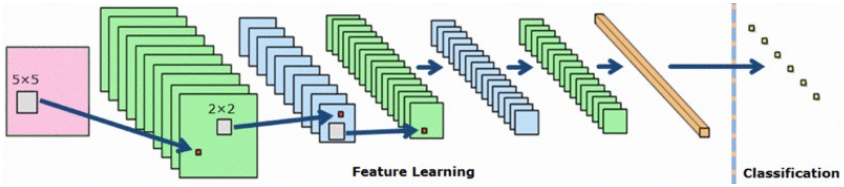
→ Re-iteratre

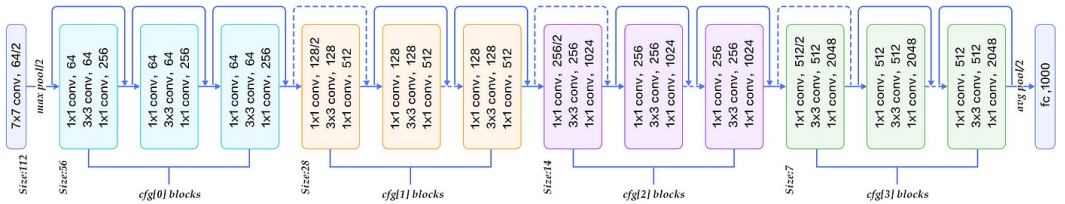
Summarize

Novelty detection with neural networks

- Recap NNs → Autoencoders
- Image AD → Intro to CNNs
- Pretrained CNNs for feature extraction
- Example AD with pretrained CNNs
- Systematic NN development







Next time

- Residual models: regression NNs
- Turbofan engine example (multivariate timeseries)
- 1D CNNs
- Wind turbine example (multivariate timeseries)

